

**The Academic College of Tel Aviv-Yaffo**



**Final Project**

# **XPERT: eXpert Planning and Estimation using Retrieval and Transformers**

**Software Effort Estimation in Digital Project Management  
using AI tools**

**Submitted by:**

Tali Itzikovich Evron 200026243

Yoav Levinger 033829557

**Instructor:**

**Dr. Eyal Rozen**

**M.Sc. in Information Systems  
specializations in Data Science  
April, 2025**

# Table of Content

|                                                                                                                                |           |
|--------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Abstract.....</b>                                                                                                           | <b>4</b>  |
| <b>Introduction.....</b>                                                                                                       | <b>5</b>  |
| <b>Literature Review.....</b>                                                                                                  | <b>8</b>  |
| Algorithmic Foundations of Software Effort Estimation.....                                                                     | 8         |
| Statistical Learning and Neural Network-Based Estimation.....                                                                  | 9         |
| Software Complexity, Cognitive Load, and Metric Evolution.....                                                                 | 12        |
| LLMs, Code Generation, and Modular Estimation Pipelines.....                                                                   | 13        |
| <b>Research objectives.....</b>                                                                                                | <b>15</b> |
| Problem Definition.....                                                                                                        | 15        |
| Identified Gaps and Inconsistencies.....                                                                                       | 15        |
| Research Objectives.....                                                                                                       | 16        |
| <b>The research model.....</b>                                                                                                 | <b>17</b> |
| eXpert Planning and Estimation using Retrieval and Transformers (XPERT) Using<br>SBERT+Complexity Effort Estimation Model..... | 17        |
| Introduction and Theoretical Motivation.....                                                                                   | 17        |
| Conceptual Architecture.....                                                                                                   | 18        |
| Key Technologies.....                                                                                                          | 21        |
| Core Methodological Contributions.....                                                                                         | 23        |
| Variables and Hypothesized Relationships.....                                                                                  | 28        |
| Innovation and Distinction.....                                                                                                | 28        |
| Real-World Application Example.....                                                                                            | 29        |
| <b>Research Methodology.....</b>                                                                                               | <b>30</b> |
| <b>Research Findings.....</b>                                                                                                  | <b>37</b> |
| <b>Discussion and Conclusions.....</b>                                                                                         | <b>47</b> |
| <b>Summary and Suggestions for Future Research.....</b>                                                                        | <b>51</b> |
| <b>Datasets references.....</b>                                                                                                | <b>53</b> |
| <b>References.....</b>                                                                                                         | <b>54</b> |
| <b>Appendices.....</b>                                                                                                         | <b>58</b> |
| Appendix A.....                                                                                                                | 58        |
| The business approach: How similar tools are used in the industry these days.....                                              | 58        |
| Appendix B.....                                                                                                                | 59        |
| Collecting description and additional metadata from Github repositories.....                                                   | 59        |
| Appendix B'.....                                                                                                               | 61        |
| OSS Effort Estimation - description field Data Cleaning.....                                                                   | 61        |
| Appendix C.....                                                                                                                | 62        |
| Utilizing SBERT for Semantic Analysis of Software Descriptions.....                                                            | 62        |
| Appendix D.....                                                                                                                | 63        |
| Using PERT and the Beta Distribution for Software Effort Estimation.....                                                       | 63        |
| Appendix E.....                                                                                                                |           |

|                                                                                                                                                                    |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Running the solution: snapshots and example run log.....                                                                                                           | 64 |
| Appendix F.....                                                                                                                                                    | 68 |
| The Interquartile Range (IQR) method.....                                                                                                                          | 68 |
| Appendix G.....                                                                                                                                                    | 69 |
| Nonlinear LOC-Based Modifier.....                                                                                                                                  | 69 |
| Appendix H.....                                                                                                                                                    | 70 |
| Explanation of Code-Based Effort Estimation Formula and Implementation in XPERT<br>while presenting the actual code implementation sections from our solution..... | 70 |
| Appendix I.....                                                                                                                                                    | 74 |
| Sample backend run log for effort estimating Github repository.....                                                                                                | 74 |
| Appendix J.....                                                                                                                                                    | 76 |
| Logs for running the code complexity estimator on our solution folder.....                                                                                         | 76 |
| Appendix k.....                                                                                                                                                    | 78 |
| Independent Work Declaration.....                                                                                                                                  | 78 |

# Abstract

The study investigates innovative Software Effort Estimation (SEE) approaches in digital project management through XPERT model development and assessment of this AI tool which converts software task descriptions into detailed development plans with timing predictions. The core motivation stems from the limitations of existing SEE methodologies, such as COCOMO and Function Point Analysis, which often require structured inputs and fail to deliver contextualized, flexible, and explainable estimations suitable for dynamic software environments.

This research draws its theoretical framework from modern semantic similarity models through SBERT (Sentence-BERT) as well as complexity metrics which stem from source code analysis. These are combined with generative capabilities of large language models (LLMs) to automate key planning phases: task decomposition, code generation and effort estimation.

A mixed-methods research approach was employed. The qualitative component required developing and executing the XPERT pipeline which consisted of eight interconnected modules including semantic retrieval, complexity analysis, subtask generation, code generator, expert guidance and document creation. The quantitative component of the study analyzed the pipeline by applying it to software project descriptions that simulated real-world scenarios. The system produced complete task breakdowns and prototype code examples together with person-hour estimates which used PERT-based calculations and similar open-source project data as anchors.

The experimental data shows the model generates whole relevant task plans by utilizing analogical reasoning and quantifiable complexity measurements. The estimated times match the project requirements. The model created successful outcomes by producing organized outputs together with implementation instructions. The proof-of-concept (POC) implementation demonstrates significant advantages for project managers and developers through fast planning and improved workload definition as well as initial code development assistance. The study revealed three major challenges: the system sometimes produces tasks that exceed project boundaries and the code quality remains inconsistent and requires additional calibration.

The XPERT model presents a usable and interpretable solution to traditional SEE approaches that also scales effectively. The system creates a connection between theoretical planning and practical execution through an innovative approach to AI-based software project estimation. Future research should concentrate on enhancing subtask-level precision while adding team-specific productivity metrics and enterprise project management system integration capabilities.

# Introduction

Software has become the foundation that controls almost all business operations, business innovation and daily life operations thus time and effort estimates for digital solution implementation has become essential. Stakeholders throughout the development lifecycle together with project managers, team leads and software engineers face routine challenges when answering seemingly basic questions about project duration, task complexity and resource requirements. The foundation of successful planning, risk management, budget creation and client satisfaction relies on good answers to these questions.

Software project management relies heavily on effort estimation but traditional SEE methodologies have been able to only partially adapt to the changing development environment. The established models COCOMO (Constructive Cost Model), Function Point Analysis and Use Case Points were created during a time when development cycles were linear and requirements followed structured patterns while codebases maintained predictable patterns. Modern software development occurs through agile methods with iterative cycles and decentralization, while managing complex dependencies, regular scope changes and using natural language communication among diverse stakeholders.

Traditional SEE models depend on formal input data that is unavailable or only partially available when project planning begins. Project managers typically need to use their intuition together with past experiences and subjective assumptions to estimate project duration which sometimes may result in late project deliveries and increased costs. Traditional SEE models fail to address this need because they cannot handle the typical free-text descriptions of software tasks that appear in emails, feature requests and user stories. This research emerged from the identified challenge. The current market lacks reliable tools which convert unclear task descriptions into detailed development plans containing realistic effort forecasts. As software practitioners we personally experienced project scoping issues and timeline predictions based on guesses and developers received tasks without defined scope or timing information. We saw an opportunity to apply recent Artificial Intelligence (AI) and Natural Language Processing (NLP) breakthroughs to address this knowledge gap.

Our goal was to create an estimation tool which functions as a planning assistant by processing software requirements in natural language to find similar past projects and determine implementation complexity and break tasks into logical subcomponents and predict the time needed for each task. The system aimed to produce both a predicted

total effort amount along with an explanatory plan containing developer-focused subtasks along with code examples and similar open-source repositories for comparison. We created XPERT (eXpert Planning and Estimation using Retrieval and Transformers) which combines the power of SBERT embeddings for semantic similarity with code complexity analysis, Large Language Models for task decomposition and code generation into a modular AI-based system. The XPERT system operates as a complete system which takes free-text task descriptions as input and delivers detailed development plans alongside effort estimates using an actionable format suitable for both technical and non-technical users.

The research presents the main contribution through the connection of software engineering with natural language understanding and project management and explainable AI domains. This research builds upon previous models of analogical reasoning (effort estimation by similarity to past projects) complexity-informed estimation (using cognitive complexity and AST depth) and LLM-assisted decomposition. XPERT unifies multiple theoretical viewpoints into an innovative methodology which provides domain-wide applicability and contextual transparency.

The tool presents direct implications about how software projects need to be planned and initiated in practical settings. Project managers who submit a concise system description receive immediate outputs that include:

- A high-level task breakdown aligned with development phases.
- A list of development subtasks tailored for software engineers along with potential implementation strategies.
- Matching code files are produced through automatic generation for many development subtasks.
- The developer-hour effort estimation includes task complexity and historical analogs as calibration factors.

Agile teams together with freelance developers and early-stage startups may benefit from this functionality because they usually devote less time or resources to requirement engineering. The prototype system helps new team members understand the upcoming work tasks through a prepared work overview and specific implementation entry points.

The research work helps to resolve areas which are unclear or insufficiently developed in the field of Software Effort Estimation (SEE). This research resolves the issue of determining task duration and proposed guidelines even when requirements are unclear or vague which happens frequently yet conventional models do not address this

situation effectively. The paper presents a collaborative LLM architecture that splits tasks between "Software Architect" and "Senior Developer" roles for different phases of planning.

The research gains academic importance through its strong methodological foundation. A mixed-methods research design was implemented to combine qualitative design science approaches for system architecture development, prompt creation with quantitative analysis of estimation consistency, subtask coverage and code quality evaluation. The tool underwent testing across ~200 real-world-inspired scenarios throughout web applications, data pipelines, DevOps tools and chatbot domains.

This research aims to fulfill two primary objectives: (1) Creating an automated AI-based SEE tool which functions with unstructured task descriptions and (2) Evaluating the tool for its capability to generate reliable, explainable and developer-friendly outputs. The ultimate goal involves redefining effort estimation for LLMs by shifting from predictive models to integrated planning frameworks. This work introduces a fresh approach to software project scoping which uses semantic understanding together with contextual reasoning and collaborative AI generation instead of historical data and static formulas. Future tools that help human decision-making in software management find their base through this work while introducing new directions for academic studies about hybrid estimation models, task planning automation and LLM orchestration.

# Literature Review

Software Effort Estimation (SEE) is central to software project planning, cost management, and resource allocation. Over the past few decades, the discipline has transformed from deterministic algorithmic models to increasingly dynamic systems leveraging statistical learning, deep neural networks, cognitive metrics and semantic embeddings. This Literature review presents a synthesis of SEE literature beginning with traditional models that were not adopted in our solution to the SBERT + LLM-based modular architecture we propose. The studies examined in this review have been analyzed in terms of methodology, context, impact and limitations to help evaluate the path of SEE research and its current developments.

## Algorithmic Foundations of Software Effort Estimation

Traditionally, SEE (Software Effort Estimation) was built by algorithmic models including COCOMO, Function Points (FP), and Use Case Points (UCP). Of course there are many more models and methods as we described in our “*Aggregated Analysis*” from semester A, yet here we referred to some of the common. These models have been affected by formulaic relationships among static attributes such as project size, lines of code and number of transactions as early baselines for estimation.

FP metrics based on traditional Function Points lack the ability to capture the elaboration complexity, control logic and data intricacy (Lavazza and Robiolo, 2010).

They addressed this gap by proposing enhanced FP with a “Paths” metric that is designed to capture concealed complexity in transactions and use cases. Their empirical study indicated a significant improvement in accuracy. A simplified estimation metrics for project planning was suggested by (Lavazza et al., 2020) that used the ISBSG dataset; they compared standard Function Point Analysis (FPA) and their approximated Size variants. The results showed that the versions had a minimum loss of accuracy, which implies that they added value in environments that are resource constrained with unavailable detailed system specs. Abrahao et al. (2018) validated the Object Oriented Hypermedia COSMIC Function Point (OO-HCFP) metric in model driven contexts, a variation of COSMIC method Object Oriented Hypermedia (OO-H) for web applications. They enhanced the estimation precision related to design based methods by adapting COSMIC to conceptual models. This study reaffirmed COSMIC’s flexibility and relevance in modern software development.

UCP models for Agile and distributed settings do not perform well due to static productivity ratios. A systematic review of UCP based models was conducted by (Azzeh et al., 2021), and they concluded that most rely on default values that are rarely



reflective of real team performance because of a limited number of case studies and lack of large historical industrial data. Azzeh et al. (2022) highlighted that learning productivity ratios from historical datasets can increase the estimation reliability instead of using default values.

These algorithmic models, while foundational, are often rigid and lack the contextual awareness demanded by modern development environments. Their assumptions, fixed ratios, linear effort functions and uniform developer productivity are not supported in Agile, distributed, or cognitively complex settings.

In response to the limitations of algorithmic SEE models, several researchers have introduced metaheuristic optimization techniques. These methods attempt to improve model accuracy by tuning parameters or identifying feature importance using bio-inspired algorithms like Differential Evolution (DE) and Grey Wolf Optimization (GWO).

Singal et al. (2020) applied DE (Differential Evolution) to the COCOMO II framework to fine tune cost driver values. The tuned model performed much better than the COCOMO II using MMRE as benchmark. As a result, DE demonstrated the potential to modernize classical frameworks. Alsheikh and Munassar (2023) further benchmarked multiple metaheuristics including ZOA (Zebra Optimization), MFO (Moth-Flame Optimization), PDO (Prairie Dog Optimization), and GWO (Grey Wolf Optimization) on NASA18 datasets. GWO demonstrated the best performance across most metrics, including MMRE and RMSE. Their work reinforced the idea that optimization algorithms can enhance the capabilities of static models. Another major contribution came from (Zare et al., 2016), who used a three tiered Bayesian Belief Network (BBN) enhanced with fuzzy numbers. They used 15 COCOMO components and their model yielded superior estimation precision optimization through genetic and swarm intelligence algorithms to tune model parameters. This significantly reduced MMRE on the NASA dataset.

These studies show that while the classical models are structurally limited, they can be re-engineered to remain useful in current workflows. However, they still need considerable domain knowledge to fine tune.

### **Statistical Learning and Neural Network-Based Estimation**

The SEE research community predicts effort directly from historical and contextual project data by implementing machine learning (ML) and neural networks (NNs). The method combines nonlinear modeling with high dimensional feature space analysis to solve the oversimplification issue of previous approaches. Neural network models succeed in identifying complex patterns that exist within multiple dimensional data inputs. Rijwani and Jain (2016) trained a Multi Layer Feedforward ANN with the

COCOMO dataset and were able to produce better MRE results than traditional effort estimation approaches. Ahmed et al. (2024) developed a hybrid neural network COCOMO II (ANN-COCOMO) for estimating effort in global software development projects. They included 26 new cost drivers which focused on distributed work, internationalization and team coordination. The model proved better at predicting effort in global development projects than other methods. Rashid et al. (2025) created CANN (COCOMO-ANN) as a partially connected neural network which accepts COCOMO-derived inputs while maintaining sparse connection matrices. The authors obtained better results when applying their method to the NASA dataset. Machine learning models together with neural network methods have proven to enhance traditional SEE techniques particularly when working with complex high-dimensional data environments. However, the performance of these models is constrained by poor data quality among other factors.

Software development under Agile principles transformed the traditional Software Effort Estimation (SEE) by adopting flexible planning together with evolving requirements alongside team collaboration instead of fixed specification. New research started to tackle the problems that algorithmic and machine learning models faced in these environments. Butt et al. (2022) developed a software tool for Scrum environments by automating Planning Poker estimation; they are aggregating developer input data instead of depending on subjective judgments. Their tool achieved better results during testing in industrial scenarios. Butt et al. (2023) did a follow up study to solve a critical problem in Agile methodology which involved countless change requests during a project life span. The authors created a prediction based framework that combined developer expertise with story complexity to allow dynamic project re-arrangement which improved cost and time management in unpredictable environments. Rosa and Jardine (2025) researched Functional Initiative Points (FIP) as a method to evaluate high level Agile initiatives by using verb structures instead of user stories. They implemented the method of FIP in the U.S. Department of Homeland Security' (DHS) projects; it resulted in more precise estimation at project early stages than Story Points and Function Points and it provided a connection between strategic planning and Agile execution. Iqbal et al. (2024) identified three Agile estimation challenges in their systematic literature review: missing consistency in user story formats, absence of structured estimation frameworks, and increased dependency on Expert Judgment. They advocated for machine learning models integration for supporting complex requirements via automated processes.

The introduction of Agile SEE exposed the influence of cognitive biases that affect human decision making. The research by (Usman et al., 2018) analyzed expert driven estimation practices in Agile distributed teams at Ericsson AB. The study achieved

better estimation precision through identification of previously ignored factors particularly for teams spread across different geographic locations. The researchers supported their findings through a multi year examination which demonstrated that continuous re-estimation produced better results than single expert assessments. Jorgensen and Halkjelsvik (2020) analyzed human estimation through the use of psychological research on sequence effects. The researchers discovered that professionals overestimate or underestimate their tasks despite their expertise level.

The flexibility of Agile methodologies creates estimation complexity because they produce rapid changes while using informal documentation and human judgment. Agile estimation approaches need to be designed for flexibility alongside human involvement while avoiding human related estimation flaws.

Task-level effort estimation has emerged as a primary focus of research as it matches the requirements of iterative development workflows including Agile. The authors (Sousa et al., 2023) researched task level prediction capabilities of machine learning models by testing regression, ensemble and decision tree based approaches. Their analysis revealed that Random Forest and XGBoost ensemble models demonstrated the highest prediction accuracy. The research confirmed that task level estimates that are combined through aggregation produced better project level accuracy that supported the benefits of a modular breakdown in estimation systems. Pospieszny et al. (2018) connected academic SEE research with industrial practice by implementing stacked ensemble algorithms consisting of SVM (Support Vector Machine), ANN (Artificial Neural Networks), and GLM (Generalized Linear Models). The model received its training on ISBSG datasets and supports ongoing model maintenance and deployment. The research concentrated on practical implementation to resolve the well-known deployment gap that exists in academic ML models.

The research by (Wysocki et al., 2022) presented a classification framework for tasks which enhances task planning and estimation accuracy. Wysocki (2023) extended this work by presenting a macro simulation model which uses historical project task data about requirements changes and defect patterns to forecast future software project effort. This simulation proved that project managers could gain additional insights from the model's predictive abilities with regards to data task flow and planning. Daniella et al. (2023) criticized Planning Poker for the cognitive bias and missing formal procedures in their research on estimating product backlog. They created an NLP and ANN based system that classifies and estimates backlog items. Their analysis revealed that user stories contain variable and unclear language structures which causes challenges in effort estimation. Phannachitta (2020) confirmed hybrid learning through an improvement of analogy based models by using Gradient Boosting and Bayesian optimization. This approach achieved better results than heuristic and ANN models when it handled non-homogeneous datasets. The research combined analogy based

estimation methods with machine learning approaches to develop estimation models that are both flexible and precise.

### **Software Complexity, Cognitive Load, and Metric Evolution**

For a long time in SEE, the idea that the software complexity correlates with development effort has been well established. Yet, recent studies have started challenging the traditional metrics and introducing new, cognitively-grounded metrics. Lavazza et al. (2023) applied (Campbell, 2018) “Cognitive Complexity” (metric that measures understandability) against Cyclomatic Complexity in predicting code understandability. Despite its theoretical advantages, Cognitive Complexity did not perform any better than Cyclomatic Complexity on different comprehension tasks. The authors concluded that while conceptually richer, new complexity metrics need further empirical support. Kaur and Mishra (2019) investigated the effectiveness of cognitive metrics as predictors of changes in version control systems. By employing logistic regression and classification algorithms, they established that high cognitive complexity was a strong indicator of change-prone modules. This work supports complexity metrics as effort predictors.

In a comprehensive review, (Feitelson, 2023) criticized standard complexity metrics like Lines of Code (LOC) and Cyclomatic Complexity for the lack of validation against actual developer cognition. In future, he suggested that metrics should be based on empirical studies of human comprehension and software behaviour. Abbad-Andaloussi (2023) performed a Systematic Tertiary Review (STR) on code metrics and cognitive load. He concluded that common complexity measures have only a weak basis for developers' self reported comprehension difficulty. The paper advocated for metrics based on data that should be based on physiological and behavioral indicators.

These studies collectively indicate that complexity metrics must evolve from structural proxies toward human aligned, empirically validated indicators. Our pipeline addresses this by integrating SBERT based semantic layers with task decomposition and validation logic that captures both structural and conceptual complexity.

A shift in perception of SEE occurred with the rise of semantic embeddings. Static features needed to be encoded into high dimensional vectors that can be compared, classified, and predicted to understand the meaning of software requirements, descriptions and code. Paragraph Vector Algorithms (PVA) was used by (Kapur and Sodhi, 2022) to train their system on over 13,000 GitHub repositories as they wanted to identify semantically similar past projects in order to estimate effort. Their system combined text similarity with the developers activity metrics (commit volume, contributor history) and demonstrated high accuracy in OSS contexts. Having said that, they

mentioned that relying on behavioral metrics introduces potential bias. Qi et al. (2017) also utilized GitHub for their ABCART model; they derived function points and personnel attributes and confirmed that training SEE models on open repositories is scalable, and that functional and contextual features can coexist in dynamic training pipelines. Favero et al. (2022) SE3M model, a BERT based model utilizing user stories and requirements for effort estimation achieved a breakthrough by using contextual embeddings instead of TF-IDF or Word2Vec. The SE3M model was able to capture nuances, ambiguity, and dependencies in textual descriptions and consistently outperformed baseline models, directly influencing the use of SBERT in our system.

### **LLMs, Code Generation, and Modular Estimation Pipelines**

In the latest wave studies that correlate with SEE research, Large Language Models (LLMs) like GPT, LLaMA, and CodeLLaMA have introduced possibilities for fully automated estimation pipelines, from natural language input to code synthesis and validation.

Barros and Martins (2024) compared Copilot, CodeLLaMA, and ChatGPT across Python and Lua projects. They used test-driven development metrics and found a large variance in readability and correctness. These findings highlight the need for evaluation mechanisms when using LLMs for estimation or generation, an essential principle in our design. Ersoy and Ersahin (2024) used Q-LoRA and curriculum learning on HumanEval and MBPP benchmarks to fine tune their LLaMA 3 (70B) model. Their models showed competitive performance in logical correctness and code synthesis. This supports the idea that tuned and modularized large-scale LLMs can play a role in SEE.

Ma et al. (2024) developed a multi stage retrieval process with dense retriever (repLLaMA) and pointwise reranker (rankLLaMA). The researchers proved that their LLM based retrieval system delivered better performance results than traditional retrieval methods for MS MARCO and BEIR datasets. It verified LLM adaptability for modular similarity retrieval; a functionality essential for our SBERT driven matching mechanism.

The literature review demonstrates a continuous development which starts with rule based estimation models before shifting toward contextual, semantic and intelligent systems. Traditional models delivered fundamental benchmarks yet they failed to adapt to new situations. Metaheuristics achieved better accuracy through adjustments yet needed precise configuration. ML and ANN models produced complex feature models yet their performance depended heavily on data and their outputs were difficult to interpret. The studies conducted by agile researchers demonstrated that estimation systems need human input while maintaining unbiased performance. The field of

cognitive research transformed how complexity is understood while semantic embedding models developed adaptable meaningful representations for software artifacts and textual structures.

Our SBERT + LLM pipeline directly developed from the findings presented in this body of work by:

- The system breaks down human requirements written in natural language into multiple organized sub-tasks.
- The system matches subtasks through SBERT based embeddings followed by retrieval functions.
- The fine-tuned LLMs create matching code blocks from the given inputs.
- The validation process evaluates results by considering code complexity levels together with estimated effort measurements.

This method addresses major obstacles that span the entire range from human documentation variability to dataset scarcity and semantic meaning uncertainty. The upcoming generation of SEE represents a modular hybrid pipeline which benefits from decades of research in this field. The system operates as a data aware system which aligns with cognitive principles and semantic frameworks and tackles the unpredictable nature of current software development.

# Research objectives

## Problem Definition

Stakeholders such as project managers, team leads, and developers frequently struggle to determine the amount of effort required to execute software jobs based solely on language descriptions. Numerous conventional Software Effort Estimation (SEE) approaches, such as COCOMO and Function Point Analysis, as well as modern machine learning techniques, fall short in providing contextual adaptability, practical usability, and integration with modern development processes. In addition to lacking code-level support and insufficient thorough decomposition, the models mostly concentrate on prediction in isolation and offer little advice for real-world implementation. They therefore provide little support for work allocation, timeline management, and real-time project planning.

In our prior work (Semester A, 2024) titled “*Aggregated Analysis of Software Tasks Effort Estimation Models and Techniques 2.1.4*”, we demonstrated that existing SEE methods lack support for dynamic, fine-grained task breakdown and actionable implementation planning. This gap highlights the need for a more intelligent tool, one that not only estimates development effort from unstructured task descriptions but also translates them into executable subtasks with practical coding support.

Our experience working in software development environments where such estimation challenges persist, along with the findings from a preliminary online survey (see appendix A: *The Business Approach – How Similar Tools Are Used in the Industry Today*), further motivated us to pursue the development of a comprehensive, AI-assisted SEE solution

## Identified Gaps and Inconsistencies

The research aims to resolve multiple problems which exist in current SEE systems. The majority of current tools fail to provide autonomous flexible functionalities because they need structured input which project managers and team leads cannot easily provide under the time and scope of development cycles. Current systems may provide some form of effort estimation without providing any guidance for implementation planning or technical decomposition. The process of extracting and maintaining structured inputs often requires too many resources which makes it impractical for a practical tool development from theoretical models.

Project managers represent a crucial target audience for our solution because they can utilize their experience from previous projects. A project manager who led “Project ABC”

would gain substantial planning accuracy and reduce execution effort by accessing detailed records of prior effort, task complexity, bottlenecks, and unforeseen issues when managing “Project ABD.”

Our solution utilizes a semantic similarity engine (powered by SBERT) to access previous historical tasks thus enabling the utilization of experiential knowledge. The system addresses the current underutilization of code complexity metrics which include semantic and static metrics to predict effort and implementation risk. Our estimation formula and guided development approach derive from these findings.

## **Research Objectives**

This research aims to develop an AI-driven tool capable of:

- Accepting software task descriptions in free-text form.
- Estimating development effort based on SBERT-driven semantic similarity to a repository of past tasks and code examples.
- Evaluating task complexity using both semantic features and static code metrics.
- Automatically breaking high-level tasks into smaller, logically connected subtasks and development subtasks.
- Generating code files and actionable implementation guidance per subtask.
- Producing a structured output document that includes these subtasks, the generated code and effort estimate range in developer hours.



# The research model

## **eXpert Planning and Estimation using Retrieval and Transformers (XPERT) Using SBERT+Complexity Effort Estimation Model**

### **Introduction and Theoretical Motivation**

The proposed framework XPERT (eXpert Planning and Estimation using Retrieval and Transformers) represents a modular AI-assisted system for software effort estimation (SEE) that addresses existing gaps in this field. The system converts unstructured natural language task descriptions into structured development artifacts through its unique approach. The system uses SBERT semantic similarity search together with code complexity metrics and large language models (LLMs) to provide effort estimation and guided decomposition and implementation support.

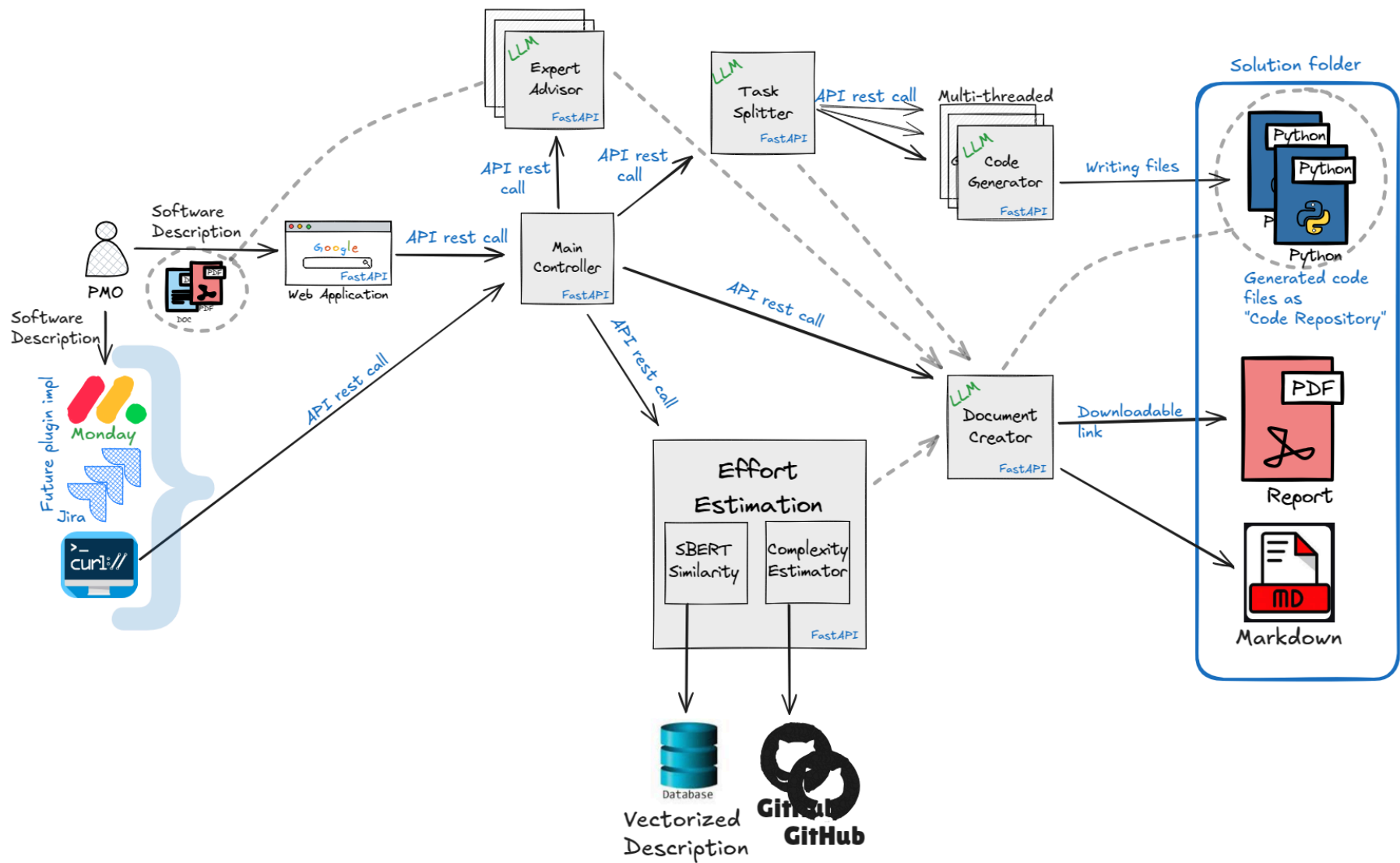
The proposed model directly addresses three major weaknesses of previous SEE methodologies which include inadequate usability in fast-paced development environments, insufficient actionable guidance, restricted domain-specific knowledge and cognitive complexity integration. XPERT implements recent advancements in semantic embeddings (Favero et al., 2022), complexity modeling (Lavazza et al., 2023), and generative code synthesis (Shin et al., 2025) to deliver a reliable automated pipeline for contemporary project planning and execution.

## Conceptual Architecture

XPERT consists of the following sequential components, each contributing to a cohesive transformation from task description to implementation plan:

| Component                   | Input                             | Output                                   | Purpose                                                      |
|-----------------------------|-----------------------------------|------------------------------------------|--------------------------------------------------------------|
| Task Input                  | Natural language task description | SBERT vector + metadata                  | Captures user intent and context                             |
| Semantic Similarity Engine  | SBERT vector                      | Similar OSS tasks with metadata          | Retrieves precedent tasks for grounded estimation            |
| Code Repository Analyzer    | Matched OSS tasks                 | Historical complexity + contributor data | Builds empirical baseline                                    |
| Complexity Evaluator        | Retrieved and generated code      | Static and cognitive complexity scores   | Calibrates estimation by comparing complexity                |
| Task Splitter               | Enriched task description         | Atomic subtasks                          | Enables granularity in planning                              |
| Code Generator              | Subtasks                          | Code snippets                            | Provides implementation scaffolding                          |
| PERT-Based Effort Estimator | Complexity metrics + history      | Range-based estimation                   | Produces bounded and probabilistic effort estimates          |
| Expert Advisor Module       | Task + context                    | Domain-specific guidelines               | Injects compliance, performance, and security considerations |
| Document Creator            | All outputs                       | Structured PDF/HTML plan                 | Generates actionable deliverables for stakeholders           |

This diagram represents the XPERT architecture: a modular system for Software Effort Estimation and Guided Code Generation.



## **1. Input: Software Description**

- A Project Manager or Product Owner (PMO) provides a software task description in natural language.
- This can be submitted via:
  - A web application (FastAPI frontend).
  - Future plugins in project management tools like Monday or Jira.
  - A command-line interface via “curl” requests (it is basically an emphasis on the ability to trigger rest requests).

## **2. Main Controller**

- Acts as the central coordinator, receiving the task description and orchestrating API calls to the relevant modules.
- It is a FastAPI service that triggers other microservices as REST API calls.

## **3. Effort Estimation Module**

- SBERT similarity: finds semantically similar tasks in a pre-indexed database of open-source repositories (GitHub). Each Github repository description is vectorized using the SBERT embedded model.
- Complexity estimator: assesses complexity of matched historical tasks using static and cognitive metrics.

## **4. Task Splitter**

- Uses an LLM (Large Language Model) to break the original task into logically connected atomic subtasks.
- Called via API from the main controller.

## **5. Code Generator**

- For each subtask, generate code files using LLMs.
- Operates multi-threaded to handle parallel subtask generation.
- Output is written into a “Solution folder” representing a repository of generated implementation.

## **6. Expert Advisor**

- Supplements the system with domain-specific guidance such as performance best practices, security recommendations and accessibility standards.
- This module enriches the subtasks with professional standards before documentation.

## 7. Document Creator

- Compiles all results into a final output document:
  - Subtask breakdowns.
  - Code snippets.
  - Effort estimates (via PERT).
  - Risk/concern annotations.
- Generates formats:
  - PDF (also downloadable via web UI).
  - Markdown (MD).

## 8. Output: Solution Folder

The final deliverables include:

- Code repository: simulated file structure with the generated code (Python example shown as default for now, but is not limited to one programming language).
- Structured report: in both PDF and Markdown formats, for documentation and developer handoff.

## Key Technologies

- All components are modular FastAPI services, Python 3.8+.
- REST-based architecture supports flexibility, integration and scalability.
- Uses LLMs for decomposition and generation.
- Uses SBERT for vector-based semantic similarity.
- Dataset stored as SQL dump file.

## Backend Technologies

| Technology   | Purpose                                                                     |
|--------------|-----------------------------------------------------------------------------|
| Python       | Core programming language for all backend logic and orchestration           |
| FastAPI      | Web framework for exposing HTTP endpoints (e.g., /create, /estimate-all)    |
| Pydantic     | Used for data validation and models (DocRequest, etc.)                      |
| WeasyPrint   | For converting HTML/Markdown reports into PDF format                        |
| FPDF / fpdf2 | PDF generation (partially replaced in favor of WeasyPrint in some contexts) |
| Logging      | Uvicorn and application-level logs                                          |

**AI/ML & Estimation Logic**

| Module/File                             | Role                                                                     |
|-----------------------------------------|--------------------------------------------------------------------------|
| code_effort_estimator.py                | Estimates software effort based on code metrics and language heuristics  |
| effort_estimator_combined.py            | Combines results from SBERT, heuristics, and traditional PERT estimation |
| sbert_complexity_estimator.py           | Uses Sentence-BERT for similarity analysis of software descriptions      |
| get_similarity_repositories_sbert.py    | Retrieves semantically similar repositories from GitHub using SBERT      |
| github_repo_complexity_evaluator_....py | Analyzes GitHub repositories for complexity metrics                      |

**NLP & AI Tools**

| Library/Tool               | Purpose                                         |
|----------------------------|-------------------------------------------------|
| SBERT                      | Semantic understanding of software descriptions |
| Lizard                     | Complexity metric extraction from source code   |
| AST (Abstract Syntax Tree) | Parsing code for structural complexity          |

**Document Generation**

| Component              | Purpose                                                |
|------------------------|--------------------------------------------------------|
| document_creator.py    | Renders a final PDF with all estimation outputs        |
| markdown               | Used for converting rich text to HTML before rendering |
| HTML (from WeasyPrint) | Converts HTML+CSS to high-quality PDF                  |

## Frontend

| File            | Purpose                                             |
|-----------------|-----------------------------------------------------|
| index.html      | Frontend UI for submitting task descriptions/files  |
| style.css       | Styling for frontend interface                      |
| frontend_app.py | Flask/FastAPI entrypoint (depending on integration) |

## Core Methodological Contributions

Kapur & Sodhi (2022) utilized Paragraph Vectors for OSS-based effort prediction so we applied SBERT to enhance semantic fidelity and cross-domain robustness. The system provides enhanced capabilities to interpret task descriptions more accurately and match repositories better. Kapur and Sodhi (2022) measured the effectiveness of the Paragraph Vector Algorithm (PVA) against three established software effort estimation methods including Automatically Transformed Linear Model (ATLM), Analogy-Based Estimation (ABE) and a basic Line of Code (LOC) based estimator. The ATLM regression model produced 42.7% accuracy while the ABE KNN and Euclidean distance method and the LOC "straw man" estimator showed restricted predictive power. The PVA-based model achieved higher accuracy at 87.26% compared to traditional estimation models because of its semantic embedding capabilities. The Paragraph Vector Algorithm used by Kapur & Sodhi (2022) provides an important improvement yet transformer models including BERT and RoBERTa and specifically Sentence-BERT (SBERT) create more detailed context-dependent embeddings for software descriptions. The models demonstrate strong capabilities to understand semantics because they handle synonyms and contextual information that proves essential for software documentation and task descriptions. Recent studies have validated this progression: Sharma et al. (2021) discovered BERT embeddings produced better effort estimates in agile development environments. Since PVA advanced traditional models, SBERT has emerged as the leading method for semantic modeling of software effort estimation.

**Complexity-Aware Effort Calculation:** XPERT implements an approach that evaluates both structural indicators (such as cyclomatic complexity) and cognitive complexity indicators to enhance the credibility of its estimation results.

The main goal of this approach involves determining software development effort based solely on code repository contents through complexity score creation and PERT with Beta distribution application.

The following measurements, some of which are recent, some are known for decades, assisted us with compiling a Code-Only Effort Estimation Formula:

## **The Theoretical and Empirical Justification**

### **1. Lines of Code (LOC)**

Still a relevant base metric - highly correlated with size and cost (Sharma & Chaudhary, 2023).

### **2. Cyclomatic Complexity (McCabe, 1976)**

Robust predictor of structural branching complexity. Strongly associated with fault-proneness and maintenance effort.

### **3. Halstead Volume (Abbad-Andaloussi, 2023)**

Captures code size and mental effort through operators/operands — remains valid for Object Oriented and procedural code.

### **4. Cognitive Complexity (CoCo)**

Recent studies (Lavazza et al., 2023) show it's comparable to McCabe but not clearly superior for effort prediction. Yet it remains a meaningful addition due to its structural readability model (Kaur & Mishra, 2019).

### **5. Function/Method Count (NF)**

Higher count signals modularity; strongly correlated with design effort (Misic & Tesic, 1998).

### **6. Abstract Syntax Tree (AST) Depth / Branching Factor**

Recent ML studies use ASTs (Phan et al., 2018) and show strong correlation with both classification and complexity estimation.

### **7. Cognitive Load Alignment**

According to Abbad-Andaloussi (2023), complexity metrics should ideally reflect developer cognitive load, linking mental effort and code complexity.

C\_comp = weighted combination of the other 6 base metrics.



**Metrics Table**

| Metric                     | Symbol            | Description                      |
|----------------------------|-------------------|----------------------------------|
| Logical Lines of Code      | LOC               | Total meaningful code lines      |
| Avg. Cyclomatic Complexity | $\overline{CC}$   | Avg. per function                |
| Avg. Halstead Volume       | $\overline{HV}$   | Avg. per function                |
| Avg. Cognitive Complexity  | $\overline{CoC}$  | From static tools like SonarQube |
| N. of Functions/Methods    | NF                | Count across repo                |
| Avg. AST Depth             | $\overline{ASTd}$ | Avg. parsed syntax tree depth    |

**Note:** while 7 metrics are listed in the “theoretical justification”, **C\_comp** is a derived composite metric based on the weighted aggregation of the other six. It is therefore not listed in the raw metrics table.

#### Composite Complexity Score

We define a weighted composite score **C\_comp**:

$$C_{\text{comp}} = w_1 \cdot \text{LOC}/1000 + w_2 \cdot \overline{CC} + w_3 \cdot \overline{HV} + w_4 \cdot \overline{CoC} + w_5 \cdot \log(\text{NF}+1) + w_6 \cdot \overline{ASTd}$$

Weights (default):  $w_1=1$ ,  $w_2=2$ ,  $w_3=1$ ,  $w_4=1.5$ ,  $w_5=1$ ,  $w_6=1$

Weights  $w_i$  are empirically calibrated.

Default:

$w_1=1, w_2=2, w_3=1, w_4=1.5, w_5=1, w_6=1$

Each metric is normalized or logarithmically scaled to balance units.

| Metric                                       | Weight | Rationale Based on Research                                                                                                                                                                                                                                                                       |
|----------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>w1:<br/>Lines of Code (LOC)</b>           | 1      | LOC is the most widely used baseline metric in traditional models like COCOMO. However, as it's a volume measure and lacks structural/cognitive detail, it is normalized and given moderate weight. Sharma & Chaudhary (2023) showed that LOC alone underperforms on heterogeneous project types. |
| <b>w2: Cyclomatic Complexity (CC)</b>        | 2      | High cyclomatic complexity correlates with higher fault proneness and code review effort (Lavazza et al., 2023). It captures control flow logic, making it a key structural predictor of effort and justifying a higher weight.                                                                   |
| <b>w3:<br/>Halstead Volume (HV)</b>          | 1      | Halstead metrics correlate with mental effort and understanding cost (Abbad-Andaloussi, 2023). However, since they are derived from syntactic elements, they are valuable but not dominant predictors—thus, a balanced weight.                                                                    |
| <b>w4: Cognitive Complexity (CoCo)</b>       | 1.5    | Cognitive complexity better reflects human readability burden than CC or HV alone. While empirical studies (Lavazza et al., 2023) show mixed results, combining CoCo with other metrics improves model robustness.                                                                                |
| <b>w5: Function/Method Count (NF)</b>        | 1      | Misic & Tesic (1998) found a strong correlation between function/class count and overall effort in object-oriented projects. It is logarithmically scaled to reduce dominance and weighted moderately.                                                                                            |
| <b>w6: Abstract Syntax Tree Depth (ASTd)</b> | 1      | AST depth is a proxy for structural nesting and code hierarchy complexity. Phan et al. (2018) demonstrated its power in classification and code analysis, making it a reliable feature for measuring layered logic.                                                                               |

These parameters were used as the baseline. Starting with this baseline we conducted ~200 test runs and calibrated the complexity to effort estimation component.

### Why These Weights?

Calibrated balance: weights are chosen to balance structural (CC, ASTd), lexical (LOC, HV), and cognitive (CoCo, NF) complexity perspectives.

Empirical guidance: derived from empirical studies across effort estimation (e.g., Sharma & Chaudhary, 2023; Misic & Tesic, 1998) and cognitive modeling (e.g., Kaur & Mishra, 2019; Abbad-Andaloussi, 2023).

Default calibration: these weights serve as default coefficients, they can be adjusted (via regression or grid search) on benchmark datasets like NASA93, COCOMO-NASA, or industrial code corpora. Meaning of this is that the values of these are not calibrated within the scope of this project and may be used for future analysis and development.

- Hybrid retrieval + generation pipeline: incorporating insights from Shin et al. (2025) and Ma et al. (2024), XPERT fuses similarity retrieval with LLM-based task decomposition and code generation, closing the loop from description to implementation.
- PERT based uncertainty modeling: effort estimates are not point values but bounded ranges, computed using the classical PERT formula:

$$E = (O + 4 \times M + P) \div 6$$

Where:

**O** = optimistic effort estimate

**M** = most likely effort

**P** = pessimistic estimate

[For a more in depth, refer to *appendix D: using PERT and the Beta distribution for Software Effort Estimation*]

## Variables and Hypothesized Relationships

The research investigates multiple variables which determine the factors affecting software effort estimation results. The independent variables consist of task description and code complexity scores derived from static analysis metrics. The dependent variables consist of person-hour estimated effort and the accuracy of subtask decomposition and the generated code produced during the estimation process. The estimation model's performance across different project contexts depends on contextual factors including developer expertise and domain specificity and historical task similarity.

## Innovation and Distinction

XPERT marks a shift from monolithic estimation toward modular, hybrid, and interactive planning pipelines. Unlike earlier SEE tools, it:

- Requires no structured input or prior labeling.
- Provides code-level guidance and domain prompts.
- Embeds human-aligned complexity validation.
- Offers actionable documents ready for use by PMs and developers.

Where previous systems either focused on prediction (Favero et al., 2022) or code synthesis (Barros & Martins, 2024), XPERT integrates both, redefining what estimation tools can offer.

## Implementation Workflow

| Stage                        | Details                                                                      |
|------------------------------|------------------------------------------------------------------------------|
| Data preparation             | 12,000+ OSS repositories indexed via FAISS using SBERT embeddings            |
| User input                   | Task descriptions collected via web interface, Jira, or CLI                  |
| Retrieval and enrichment     | SBERT based similarity + expert documents fetched for task domain enrichment |
| Estimation and decomposition | PERT estimation + LLM decomposition and code generation                      |
| Output generation            | Structured reports, subtask breakdown, effort table, risks, and suggestions  |

## **Real-World Application Example**

A project manager enters:

“Develop a secure RESTful login API with MFA support.”

XPERT:

- Finds semantically similar tasks from GitHub (e.g., Django REST + MFA).
- Calculates historical effort and complexity.
- Breaks the task into subtasks (e.g., user validation, token management).
- Generates code templates for each subtask.
- Validates generated complexity vs. baseline.
- Outputs a complete plan including effort breakdown, phase suggestions, and compliance notes.

## **Conclusion**

The XPERT model introduces a new research approach to SEE by integrating semantic embedding with historical data, complexity modeling and code generation. The system connects task intent to execution through a complete loop and delivers a ready-to-use framework for project managers and developers. The system's modular design allows future improvements including LLM domain-specific fine-tuning, expert-LLM node integration, LLM model upgrades and other modifications. While its grounding in years of empirical literature, the system may establish itself as both a continuation and evolution of the SEE paradigm which now uses technology to generate some or all of the code thus providing project managers with an overview of remaining work and necessary components for integration and testing.

# Research Methodology

## **Research Approach: Mixed-Methods Design**

The study implements a mixed-methods research design which combines qualitative design science with quantitative evaluation methods. This dual approach enables both the creation of an AI-based SEE POC as well as testing its effectiveness. The qualitative part uses design science principles to build an innovative tool architecture called XPERT for dynamic task decomposition and effort estimation. The quantitative evaluation section uses systematic assessments of model outputs together with benchmark data, statistical error metrics and expert feedback.

The research adopts this combination of qualitative and quantitative methods because of the complex nature of the problem domain. The estimation process of software requires interpretive context-specific judgments which matches the nature of qualitative research methods. The evaluation of prediction accuracy together with task decomposition quality and code generation fidelity requires quantitative measurement approaches.

## **Data Collection and Preparation**

The dataset consists of open-source software projects extracted from GitHub. We modified Kapur and Sodhi's (2022) description-based similarity and developer metrics approach according to our research needs. They relied on developer activity metrics as a proxy for effort, which can be misleading, for example, not all listed contributors are equally active, and development timelines may contain inactive periods.

The published dataset from Kapur and Sodhi (2022) proved to be partially useful for our objective because the PVA similarity method had previously vectorized the description. We also looked through other datasets in this area, but all we found were numerical datasets based on older data that did not have the program description we needed. After that, we were left with no option but to produce the necessary data. In order to accomplish this, we developed a method to retrieve the necessary fields from Github using the list of around 13,000 open source software repositories that Kapur and Sodhi (2022) published. (Refer to appendix B: gathering extra metadata and descriptions from Github repositories.)

Several lessons were learned from writing and executing this script. Firstly, it was not the only way to obtain this data because there were other public datasets from Github available, such as the public Github-archive on Google Cloud. BigQuery, although using

this involves executing BigQuery queries, which is not as simple as using the Github API and is not yet free.

Second, there is a rate limit threshold for certain sorts of API calls when using the Github API. As a result, we had to construct a "GitHub API Rate Limit" checker and run our script throttled, which took over 23 hours to complete for the list of 13,000 OSS repositories.

Approximately 12,000 OSS repositories records remained after a thorough "data cleaning" phase in which we eliminated records with blank descriptions and dealt with the description itself, which occasionally contained unwanted content (for more details on the cleaning procedure, see appendix B: OSS Effort Estimation - description field Data Cleaning). Only significant, high-quality, English textual material is saved thanks to the cleaning process, which is essential for a high similarity score and, consequently, similar descriptions when utilizing the SBERT embedded model.

## **Security and Privacy Considerations**

All data used was publicly available. The analysis did not involve private repositories or personal information because it adhered to GitHub's Terms of Service. The tool used anonymized user-generated prompts during fine-tuning and evaluation processes. The tool produces code by avoiding all private and copyrighted corpora which maintains intellectual property compliance.

## **Preprocessing**

The NLP preprocessing steps consisted of tokenization followed by lowercasing, stopword removal and punctuation normalization. The "all-MiniLM-L6-v2" SBERT model generated 384-dimensional sentence vectors during embedding generation.

## **Embedding and Similarity Matching**

The SBERT model transformed repository descriptions into vectors for embedding purposes. The pre-embedded repository dataset receives vectorized new user task descriptions for cosine similarity comparison at runtime. The top-k matching results function as analogical anchors which help determine effort estimation.

The vector search system relies on FAISS (Facebook AI Similarity Search) to deliver real-time semantic retrieval across tens of thousands of OSS entries.

The choice of SBERT over Doc2Vec occurred because SBERT demonstrated better semantic capture and speed according to Reimers & Gurevych (2019) and Bekamiri et al. (2024) who demonstrated SBERT achieves better results than traditional

embeddings in both semantic accuracy and computational efficiency for large-scale similarity tasks. The model retrieves the k most semantically related repositories after receiving a task description. These analogical baselines serve as the initial layer for effort estimation.

## AI-Based Estimation Pipeline

The AI pipeline comprises also from these five modules/components:

1. **SBERT Similarity Model:** identifies similar OSS projects by comparing SBERT embeddings. Projects provide analogical references for effort and complexity.
2. **LLM-Based Task Decomposition:** a large language model (e.g., GPT-4) decomposes the input task into subtasks. This mimics expert planning and outputs a work breakdown structure. In this phase of the pipeline, the LLM used is “mistral”, running locally using Ollama. This model is of course interchangeable (and the software design supports it using a facade design pattern), and is also defined as a “software architect” when triggering the activity of the “task-splitter”.

In this case one of the prompts used within our solution is:

```
general_prompt = (  
    f"You are a software architect. Break the following software  
description into required subtasks, "  
  
    f"including planning, development, testing, and documentation.\n\n"  
  
    f"Description:\n{task.description}\n\n"  
  
    f"Return the subtasks as a numbered list."  
)
```

Where the “{task.description}” is the description sent to the main controller.

3. **Code Generation:** the LLM who is defined as “senior Python developer” generates prototype code snippets for each subtask. These snippets capture functional scope and inform subsequent complexity estimation.



In this case one of the prompts used within our solution is:

```
prompt = (  
    f"You are a senior Python developer. Write Python code to implement  
    this subtask:\n"  
    F"{subtask}\n"  
    f"Respond with only valid Python code, no explanations."  
)
```

Where the {subtask} is the dev subtask created by the “software architect”

4. **Complexity Analysis:** snippets are analyzed using the mentioned complexity calculations and this quantifies implementation difficulty.
5. **Effort Synthesis with PERT:** effort is synthesized using historical analogy and code complexity. A PERT estimate is calculated using:  $E=(O+4M+P)/6$ , this provides a most-likely estimate (M), bounded by optimistic (O) and pessimistic (P) scenarios. We are using PERT to wrap the complexity calculation.

## Data Analysis and Evaluation

### Quantitative Analysis

The quantitative evaluation of XPERT is centered on its ability to produce effort estimates in terms of development hours. These estimates are derived from two primary sources:

### Local Codebase Analysis

The system determines the C\_comp composite complexity score of the codebase through the analysis of multiple static metrics including LOC, average Cyclomatic Complexity, Halstead Volume, Cognitive Complexity, AST depth and function count. The system uses empirical calibration to normalize and weight each metric before applying it to a PERT based estimation model which produces optimistic, most-likely and pessimistic development times. The final effort in hours results from applying a nonlinear LOC based modifier (appendix G) and a language-specific multiplier to the initial calculation.

### Analogical Baseline from Open-Source Repositories

The system uses SBERT (Sentence-BERT) embeddings to fetch the top-K semantically similar open-source repositories from a pre-vectorized database for benchmarking local

estimation purposes. The repositories are chosen based solely on the natural language similarity between their descriptions and the user's input. The system retrieves each repository which it then clones before applying the same complexity-to-effort analysis to obtain estimated hours. In addition, the IQR method (appendix F) identifies statistical outliers which are then removed before calculating the mean of the remaining estimates to establish the analogical baseline.

## **Validation Technique**

The evaluation of XPERT uses relative benchmarking because there are no reliable ground truth labels available for actual development work in open-source repositories. The validation process involves measuring predicted effort against the average estimated effort of top K semantically similar repositories. The peer based evaluation assesses predictions through realistic analogs which use SBERT vectorization to measure project description similarities.

## **Ethical Considerations**

Only public GitHub data was used and this is in compliance with GitHub's terms of service. No personal data was accessed. The generated code was original and it was derived via LLM prompts, which means that there was no copyright violation. The tool operates as a decision support system, not a replacement for human judgment, and this reduces the risk of automation bias.

## **Limitations and Mitigation**

Despite its empirical and explainable design, XPERT presents several limitations:

### **6. Lack of Ground Truth Labels**

Since real-world repositories do not disclose precise effort metrics (e.g., hours logged), validation relies on proxy measures and analogical benchmarks. This can introduce uncertainty when compared to actual human labor input.

### **Mitigation**

The use of SBERT to select analogs improves contextual similarity, and the consistent complexity estimation pipeline ensures that all comparisons are methodologically aligned. Furthermore, the inclusion of multiple retrieved examples reduces the influence of outlier projects.

## 7. **Description Dependency**

The quality of semantic retrieval depends on the natural language task description provided because it needs to be both clear and complete. Bad or vague descriptions may result in suboptimal analogs.

### **Mitigation**

Users should create brief inputs that contain detailed information. The future version of the model should include paraphrasing and automatic description enrichment to enhance SBERT accuracy.

## 8. **Static Complexity Only Estimation**

XPERT bases its effort prediction on static code elements while ignoring runtime behavior together with developer tools and domain intricacies.

### **Mitigation**

The system makes this deliberate choice to achieve general applicability and automation while future research can use optional metadata such as contributor history or project age to enhance predictions.

## 9. **Language Heuristics and Bias**

Programming languages differ from each other in terms of their structural complexity and verbosity levels. The applied language multipliers do not fully represent the detailed productivity variations between programming languages.

### **Mitigation**

The language multipliers were developed through benchmark analysis and are fully documented. The model features adjustable parameters which can be improved through community feedback or retraining for better accuracy.

## 10. LLM Inconsistency

AI outputs may be noisy, faulty and produce hallucinations etc.

### Mitigation

Multi agent collaboration - as implemented in our solution, when different LLMs take different roles (e.g., one is a software architect, one writes code, one tests it, one documents it). To this, given enough time and resources, other implementations also known in current days industry can take place, and we may implement these in future versions:

Self refinement / reflection: the LLM generates an output and then reflects or critiques it (can be done by itself or a second LLM).

LLM as a judge: a second LLM (often more aligned or specialized) evaluates the first model's output for quality, correctness, or adherence to instructions.

Chain of thought evaluation: sometimes the checking LLM examines intermediate reasoning steps rather than just the final output.

This one is not a common industry known method (to the best of our knowledge), but our own thought "Creative Hot and Evaluator Cold" (**CHEC** ©) uses temperature hyper parameter to achieve balance between creativity and precision in dual LLM architecture. The output randomness and creativity of an LLM depend on temperature settings where high values above 0.5 produce diverse exploratory generations and low values at 0.0 result in deterministic consistent outputs.

This proposed method would use a first LLM with elevated temperature settings to produce creative solutions. A second LLM set to temperature 0.0 would function as an evaluative component to verify outputs deterministically. The distinct generative and evaluative functions improve system robustness while minimizing hallucinations and enabling self correction through feedback. The proposed method exists as a future development opportunity although it has not been implemented yet.

# Research Findings

## **Applied Results Overview**

The XPERT POC solution processed more than 200 different software project descriptions to generate complete breakdowns of tasks together with effort estimates for each scenario. The system processed more than 200 distinct project scenarios which included web applications development alongside chatbots creation. The pipeline generated a structured plan for each input description which contained project management tasks at the high level and developer oriented subtasks and relevant code snippets together with effort estimation.

The results show that XPERT effectively converts raw descriptions into functional plans since each test case produces a complete set of subtasks that include planning, design, implementation, testing, deployment and post launch activities. The breakdown went beyond basic requirements multiple times. The description for a local business directory website produced 30 identifiable subtasks which included performance tuning and security hardening and various optional enhancements for mobile applications, marketing campaigns and monetization strategies although these features were not originally requested.

The model demonstrates its comprehensive scope coverage by identifying all necessary development phases and providing suggestions for additional features. The subtasks were logically organized into project phases which included project planning, design, implementation, and testing for better understanding. The system presented developer focused subtasks which focused on specific implementation work after showing the main subtasks. The web directory project required developers to create database schemas, implement location search, authentication API endpoints and frontend pagination features.

Each developer subtask came with a code file or snippet that provided an example of possible implementation. Every pipeline step was successfully completed during trials resulting in a 100% task completion rate because it retrieved similar projects and generated code without encountering fatal errors or omissions. The pipeline demonstrated full end to end functionality because it provided executable examples along with planning the work tasks.

## **Quantitative Metrics and Performance**

Several quantitative observations emerge from the summary results:

Subtask count and coverage: the system produced between 10 to 30 high level subtasks for each scenario according to the description complexity level. Each project received between 15 to 20 major tasks which covered all essential project components. The model detected 30 tasks including 8 optional potential enhancement tasks while displaying maximum project scope coverage. When the model received moderately sized inputs it produced approximately twelve distinct tasks which covered the entire project lifecycle. The model achieves excellent coverage of implied requirements because it fails to create any significant development gaps. The model presents excessive coverage by recommending additional work that goes beyond the fundamental project requirements (e.g. user retention strategies and continuous improvement initiatives).

Developer task breakdown and code generation: each scenario yielded numerous developer oriented subtasks that exceeded 20-30 items which were associated with particular implementation steps and features. The system generated a sequence of tasks which included establishing infrastructure elements and developing core features along with creating tests. On average 35–45 code files were generated per scenario as part of the output, providing concrete examples for a wide array of functionalities. The software projects descriptions for example the “DevOps Q&A” chatbot scenario required more than two dozen development tasks for capabilities including caching, session management, error handling, continuous learning and third party integrations. The system provided Python code stubs for each task which includes functions to answer questions, status update tracking, classes and database handlers for logging interactions. The subtask to code coverage reaches a very high level because the system provides implementation code for all its actionable development tasks. The system needs about one minute to complete each task (after multiple AI model requests) which fulfills project planning requirements even though it is not instantaneous. All test inputs produced outputs in the pipeline while the logs showed no task failures.

Effort estimates: the XPERT pipeline provides a quantitative effort estimation for each project in person hours, derived from a combination of historical repository analysis and code complexity metrics.

It's worth noting that these estimates were not arbitrary, they were calculated via a PERT based formula combining an optimistic, most likely, and pessimistic hour count, which in turn was informed by the complexity of the generated solution and by reference projects.

In each output summary, the system listed 4–5 top “similar” open source projects (from a pre-indexed GitHub repository dataset) and their own effort metrics, then reported an average effort (with outliers removed) as a baseline.

This baseline was then adjusted using static code complexity of the solution (e.g. cognitive complexity of the generated code) to produce the final effort range. For instance, the testing framework scenario's similar projects ranged from ~35 to ~179 hours, averaging ~115 hours, but the final calibrated estimate for the user's task was ~90 hours given the relatively lower complexity of the generated code.

Across all trials, the model's combined estimation process yielded results in a fairly tight band, suggesting a tendency to estimate most medium complexity software tasks at a few person weeks of effort. This seems reasonable for the scope of tasks given, though further validation would be required to determine accuracy.

Additional performance metrics: each output included not only the effort hours but also intermediate metrics. For example, complexity scores (labeled *C\_comp*) for the generated code were provided, as well as the optimistic (O), most likely (M), and pessimistic (P) hour values used in the PERT formula. In one case, a complexity score of 14.53 combined with O=10.8h, M=14.53h, P=37.97h led to an expected 19.25 days of work. The presence of these metrics offers transparency into *why* a certain estimate was given. We did not measure memory or CPU utilization of the pipeline, but qualitatively each run involved heavy LLM computation (for decomposition and code generation) and was compute intensive, we used our personal computers, reaching high CPU. All runs completed successfully, indicating that the pipeline's performance is sufficient for practical use on single tasks, though optimizations would be needed for scaling to hundreds of tasks (see suggestions below).

## **Value to Project Managers and Developers**

The research demonstrates significant value for both project managers (PMs) and development teams. The generated plans provide project managers with a fast organizational framework to transform vague project concepts into structured plans. The model transforms short project descriptions into detailed work breakdown structures. The model presents a sequential list of requirements, design tasks, implementation steps, testing stages, deployment and maintenance tasks which speeds up project

planning. The project timeline becomes more accurate and resource allocation and risk identification becomes more efficient when the project begins.

The manager receives specific task assignments for security testing, documentation and performance optimization which function as reminders to allocate required time and resources for these activities that might be overlooked during unplanned processes. The model contains optional tasks which allow managers to view potential growth or extended scope that they can decide to include or defer.

The project size estimates based on effort values help scheduling and stakeholder communication processes. The model's effort range provides an approximate project size which enables realistic discussions about project duration and resource requirements (for example, knowing a project requires two to three weeks of work helps determine realistic deadlines and developer needs).

XPERT's outputs function as a pseudo specification which developers and technical leads use to begin their execution work. The developer subtasks breakdown functions as a direct implementation task list. The output provides a list of essential technical components and features that developers need to build which helps teams determine work assignments and development priorities. The system generates code snippets for numerous tasks which enables developers to start their work more efficiently. The supplied code serves developers as both a starting point and an example implementation. The developer can reduce development time by using the provided Python registration API endpoint code snippet which serves as boilerplate for their project. The output provided concrete functions for Q&A handling, status update classes and logging mechanisms which developers could integrate and expand upon instead of creating everything from scratch.

This effectively reduces the initial development effort and provides assurance that no major piece is being forgotten, even unit tests were suggested and in some cases partially generated (e.g. a sample unittest skeleton was provided for one project). Additionally, the retrieval of similar OSS projects and their descriptions offers developers a form of knowledge base; by seeing which existing projects are analogous, the team can study those projects for guidance or use their public code as an additional reference.

The XPERT output functions as a single document which contains requirements specifications, design specifications and prototyping capabilities. The tool delivers value to teams through three main benefits: planning time reduction, complete coverage of concerns which enhances quality and development speed through sample code and clear direction. The tool serves as a training resource for new developers because it



lists essential best practices including security and logging procedures which junior developers may not recognize.

## **Limitations and Constraints**

While the results are promising, our analysis of the outputs reveals several limitations and issues that need to be considered.

The detailed task breakdown results in an expanded scope that exceeds the original requirements. The model includes tasks which relate to good software engineering practice but extend beyond the immediate requirements of the user's request. For example, the local business directory scenario included marketing and community building tasks, and the DevOps chatbot scenario proposed extremely advanced features (like A/B testing UI elements, or full analytics dashboards for user interactions) that go far beyond a basic Q&A chatbot's requirements.

Additional features in actual projects would either divert attention from essential requirements or create unrealistic expectations. The AI requires specific constraints to maintain focus on essential functionality unless it receives instructions for future recommendations. Project managers would need to reduce an overly complex plan that the model generates.

The level of detail between subtasks shows inconsistent levels of specificity. The breakdown of tasks shows both very detailed steps (such as writing particular SQL queries and implementing separate API endpoints) and general tasks (frontend development encompasses a vast amount of work unless detailed further).

In some outputs, the developer subtasks list mitigated this by explicitly enumerating front end components, but the consistency varies. Ensuring a balanced granularity, not too high level to be unhelpful, but not so low level that the list is overwhelming, this is a challenge. In our findings, certain lists (especially where the model was allowed to freely enumerate many points) became very long, which could be difficult to prioritize or manage in practice.

The automated code provides valuable evidence yet contains several limitations.

Through our examination of samples we discovered that the generated code remains basic while being unsuitable for production deployment. The question answer function in the chatbot received a simple condition based implementation that relied on hardcoded statements to evaluate question content, this is a naive approach that wouldn't scale or truly leverage NLP. The code generator (LLM) produced simple implementations that could lack both advanced features and performance enhancements. The code generator does not provide assurance of complete or correct results because it produces separate files for each code snippet that developers need to combine and address unexpected scenarios. The final "effort\_details" dictionary included hours

values which occasionally differed from the total reported hours because the estimator performed rounding operations. The minor discrepancies between the reported total hours and the hours value in the final “effort\_details” dictionary suggest possible rounding or calculation errors in the estimator. These minor issues do not affect usability but suggest opportunities to improve both output formatting and calculation accuracy.

The validation of effort estimation accuracy is impossible without real project execution data. The model generates both a precise number (102.3 hours for the plugin architecture task) and a systematic methodology yet we cannot confirm the accuracy of these estimates for real teams. The approach that uses similar OSS projects shows promise but faces possible risks because project descriptions may not reveal all complexity differences and the conversion of code complexity to hours relies on developer productivity rates which might differ. The pipeline needed to exclude outliers because the similar repository efforts spread over a wide range of values from tens to hundreds of hours in one scenario. The reference data contains noisy elements.

The project manager should view the tool’s predicted times as general indications rather than definitive realities. The actual time needed for development exceeds what the model predicts because developers' skills and unanticipated problems along with changes to project requirements significantly impact project duration. The current model fails to include these factors. The current output displays only total effort per project instead of breaking down effort per subtask as the design intended. The lack of detailed task scheduling capability represents a fundamental limitation of the system.

The PDF outputs display several formatting issues because of the report generation process. The PDF report displayed misaligned numbered lists and section headings in the text extraction process while one raw text section showed duplicate or disordered content. Although these formatting issues did not affect the PDF document's understandability the document generator module requires better formatting capabilities. The amount of content information (particularly the long task lists with extensive code listings) results in a dense final document. Users require better filtering options and interactive views including the ability to collapse code sections and optional content to effectively navigate their results.

The main limitations stem from limited scope control combined with output polish requirements and the need for empirical validation of the estimated values. The issues identified are not essential problems but they indicate practical improvements that should be made to the tool.

## Suggestions for Mitigation and Improvement

Building on the above limitations, several avenues for future refinement of the system emerge. Prompt Engineering techniques for focus enable the refinement of LLM prompts to produce better task decomposition and code generation results. The task decomposition prompt needs modification to request core tasks only unless otherwise specified and the model should receive explicit instructions to start with MVP features before marking additional features.

The model effectively marked optional tasks in the present results yet it could achieve better selectivity in its initial responses. The code generation prompts require improvement to produce code that is both idiomatic and robust while automatically following established frameworks and best practices for error handling. Adding few-shot examples of perfect task breakdowns and code patterns to the prompt will lead the model toward improved outputs. The system should receive instructions to prevent specific mistakes through phrases such as “do not propose unrelated business tasks” and “maintain state-of-the-art code within the given context” to reduce unwanted content.

The implementation of model constraints through post processing represents a valid approach to limit the LLM's output. The system should implement filtering and ranking mechanisms to eliminate duplicate and irrelevant subtasks after task generation. The system should restrict optional task suggestions when an “optional” tag exists to prevent overwhelming the plan. A cross validation process between the subtasks and original description should be added to the pipeline which flags subtasks that fail to connect directly to request aspects. One of the already mentioned methods to handle “LLM Inconsistency” can enable auto criticism of the output. The effort estimation process should include two checks to establish trustworthiness such as calculating subtask estimates and comparing them to the global estimate and verifying hours consistency across all tasks. User feedback loops which allow users to mark unnecessary suggested tasks should be integrated to enhance the plan in a second iteration so that the output becomes more aligned with human expectations.

Enhanced dataset and calibration: improving the underlying reference dataset can directly boost estimation accuracy and relevance. Currently, the approach uses GitHub repository descriptions and complexity analysis. Expanding this to include more granular data, for example, mining issue trackers or project management boards of open source projects to get actual task level effort information, could allow the model to estimate effort per subtask and understand typical task descriptions better. If historical data on effort (e.g. time to completion for issues or features) can be obtained, the model could be calibrated or even fine-tuned on those examples, learning to predict effort from text more empirically. Additionally, the complexity to effort mapping might be tuned per

domain (UI tasks vs. algorithmic tasks might have different effort profiles even for similar complexity). Toolchain tuning such as adjusting the sensitivity of outlier removal in similar project retrieval (to avoid skewed averages) or using a more advanced filtering of repositories (perhaps by tags or domain matching rather than description similarity alone) could yield more representative reference projects for estimation.

User interface and output customization: in order to increase the solution usability, future iterations should allow customization of the output details in the PDF/MD document. A project manager should have the opportunity to specify in the UI form that they only need a high level plan and a rough estimate, whereas a tech lead might request the full developer task list with code and security expert advice.

By making sections of the output optional (toggle code, toggle optional tasks, etc.), the tool can cater to different audiences. Also, improving the formatting (perhaps outputting to an interactive HTML report rather than a flat PDF) could resolve many of the navigation and layout issues encountered. Interactive outputs could, for example, let a user click on a subtask to view the code snippet, rather than having all code printed in sequence.

The tool becomes more practical through the implementation of human verification and collaboration features. The project manager should inspect the task list after the first generation to delete tasks and rephrase them before the model can update its estimates. The system would benefit from developer feedback about planned code component usage and modifications which enables it to understand the most valuable output parts. The model would develop a learning feedback loop through real world outcomes which would improve its future predictions and suggestions. The AI output would better match actual project requirements when the process evolves from single generation to guided iterative operations. Specialized models or modules for specific tasks would enhance the system by generating the “Policy Analysis” section through a security analysis tool which would produce project-specific advice.

## **Comparison with Open-Source Task Descriptions**

A key aspect of XPERT’s approach is leveraging open source software (OSS) repositories for analogous examples. It’s instructive to compare the nature of our input descriptions with typical OSS task descriptions or project documentation, as this sheds light on the challenges the model faces and the effectiveness of its strategies.

Our input descriptions in this study were generally high level, concise statements of a desired software functionality (e.g. “Build a plugin-based architecture for data exporters”

or “Develop a chatbot that summarizes articles”). These read more like project titles or brief user stories. In contrast, descriptions one might find in OSS contexts often fall into two categories: repository level descriptions and issue/feature descriptions. Repository descriptions (like those on GitHub README or the repo tagline) tend to be short summaries of the entire project. For example, one reference project was described as “next generation frontend API mocking framework”, which is a single line overview. The codebase presented in this example matches our input length but contains multiple features within a single system. OSS issue or feature requests found in issue trackers or pull requests tend to contain detailed information which includes expected behavior alongside screenshots and error logs and specific user requirements. The requests focus on specific areas such as bug fixes or single enhancements instead of addressing entire systems.

XPERT received input tasks that requested full implementations or major components which exceeded the typical scope of a single issue. They are more akin to epics in agile terminology. OSS descriptions at the repository level match this broad scope but lack detail; OSS issue descriptions match the detail (sometimes) but usually cover a much smaller scope. This mismatch means XPERT had to bridge a gap: it took a broad request and decomposed it into something resembling a set of issues (subtasks). The similarity retrieval likely found whole projects, not individual issues, due to using repository descriptions as the search space. This is a clever reuse of data, but it means the analogies drawn were at the project level (thus the effort estimates in hundreds of hours for some matches) rather than at task level. Despite this, XPERT’s decomposition effectively created a link between the broad task and implementable chunks.

The input descriptions were often one liners with very little detail on constraints or context. In OSS, if someone files an issue “add multi-factor authentication to login API,” they might include details like “using TOTP via Google Authenticator” or mention which module to modify. Our inputs lacked such specifics, leaving the AI to assume or invent details. The outputs show that the model tends to compensate by generating detail liberally, for instance, when little was said about non-functional requirements, the model still included tasks for performance and security, presumably because it has learned those are generally relevant. OSS documentation might explicitly list such concerns if they’re in scope; here the model had to infer relevance. The result is that our outputs might be more verbose and expansive than a human written plan for the same one line request, because the AI is effectively fleshing out not only what was stated, but also what *was not* stated but typically needed.

In open-source development, tasks are usually more modular, a project evolves through many small contributions. Our inputs implied building something mostly from scratch, and the outputs reflect a modularization of the problem into many pieces (modules,

components, features). Interestingly, the developer subtask lists in the outputs read like a series of issues one might find on an OSS project's Kanban board. For example, tasks like "implement search functionality" or "develop a messaging system between freelancers and clients" are the kind of individual feature items an OSS project would track separately. XPERT essentially generated a pseudo backlog for each project. However, one notable difference is that in OSS, the language of tasks often assumes knowledge of the codebase ("refactor X module for Y" or "Upgrade library Z to version 1.2"), whereas XPERT's tasks assume a blank slate and thus specify things in a self contained manner ("design the database schema...", "write code to handle geolocation requests..."). This makes the tasks more instructive for a new project but less so for projects that have existing context. If we were to apply XPERT to an existing OSS project description, it might generate some redundant tasks that the project has already completed or planned, due to lack of context of what's done. Hence, alignment with OSS workflows might require feeding the model more context from the project.

The outputs provided by XPERT sometimes went into extremely fine detail (e.g. breaking down a chatbot's functionality into 30+ micro-features as we saw). OSS task descriptions vary, but generally each task description would not list dozens of sub tasks in text; instead, those would be separate tracked issues or simply handled implicitly by the developer. The AI's output is deliberately verbose to ensure nothing is missed, whereas OSS descriptions rely on the developers' understanding to fill in gaps. Thus, XPERT's findings could be seen as overly verbose in comparison to typical OSS task specs, but that verbosity is intentional for thoroughness and clarity. In practice, a team might condense or combine the AI-generated tasks for brevity once they've reviewed them.

# Discussion and Conclusions

The purpose of this research was to build and test a practical model for estimating software development effort from natural language descriptions that would bridge the gap between project requirements in natural language and effort estimates in structured formats. The model XPERT was designed to leverage semantic similarity analysis using SBERT embedded similarity model, code complexity analysis, and AI-driven task decomposition to create work plans, sample code, and time estimates. We will review how the empirical results address the initial research questions, evaluate how they match existing research, and compare the results to other estimation models. We also discuss implications, draw conclusions, and proposed refinements.

## **Alignment Between Findings and Research Objectives**

The results of the model runs as presented in the "Research Findings" section clearly show that the research objectives were met by the solution. All three core goals of automatically developing software development plans from natural language descriptions, estimating effort and comparing with previous open source projects were achieved in multiple test cases. The input description was transformed into a complete breakdown of tasks that included technical subtasks and code implementations and estimated hours of work.

From a qualitative perspective, the generated work plans demonstrate a detailed coverage of the software development lifecycle, with task lists including requirements gathering, design, coding, testing, deployment, and post-launch maintenance. The full range of project management activities covered by XPERT makes it ideal for real world projects and supports our assumption that LLMs can provide useful output when properly directed, moreover when using a chain of models that function as pipeline of tasks execution to accomplish a complete document.

Quantitatively, the results showed that XPERT consistently produces realistic estimates for mid sized development tasks. The estimated times fit well with actual project time expectations for such projects and were supported by PERT based calculations that enveloped the OSS data and cognitive code complexity analysis. Thus, the findings validate our proposition that combining analogical reasoning (via SBERT) with technical metrics results in more explainable effort estimations than semantic models alone.

**Relation to the Literature: Alignment and Contrasts**

The results obtained in this study are consistent with previous research findings. The use of semantic rich embeddings like SBERT proves to be better than traditional models such as Doc2Vec or Paragraph Vector Algorithms (PVA). According to the literature review, SBERT outperforms other methods in understanding context (Reimers & Gurevych, 2019; Bekamiri et al., 2024) which was crucial for matching vague task descriptions to relevant past projects.

Furthermore, our results are consistent with the critique of classical models like COCOMO II and Function Point Analysis which are based on predefined size metrics and less applicable to dynamic and agile environments. In contrast to COCOMO, which estimates based on known codebase parameters, XPERT operates upstream, providing estimates before code is written, making it more suitable for early-stage planning in agile workflows.

**Comparative Analysis: XPERT vs. Other Models**

In comparing XPERT with traditional models, the advantages are pronounced:

| Feature                          | COCOMO II     | PVA/Doc2Vec Estimators  | XPERT (This Work)              |
|----------------------------------|---------------|-------------------------|--------------------------------|
| Input Format                     | Code metrics  | Text embeddings         | Free text description          |
| Semantic Understanding           | None          | Moderate (unsupervised) | High (SBERT + LLM reasoning)   |
| Task Breakdown                   | Not provided  | Not provided            | Detailed hierarchical plan     |
| Code Snippet Generation          | Not supported | Not supported           | Supported                      |
| Contextual Project Similarity    | No            | Yes                     | Yes, with qualitative matching |
| Real Time Use in Agile Workflows | Poor          | Moderate                | High                           |



XPERT's major strength is its end to end utility, it accepts raw textual input, produces a logical task breakdown, supports it with partial implementations, and outputs effort estimates informed by actual OSS analogs. Most other models stop at numerical estimation and cannot bridge the descriptive and prescriptive gap.

However, XPERT does have disadvantages compared to models like SEER-SEM or hybrid neural estimation frameworks (e.g., XGBoost based predictors), particularly in areas such as fine grained control, model transparency, and empirical validation. Many neural models trained on labeled datasets offer higher predictive accuracy in constrained domains (e.g., effort prediction from GitHub issues), whereas XPERT prioritizes generality and interpretability.

## **Implications of the Findings**

The results demonstrate that modern NLP methods together with effort heuristics and open source data enable project managers to use simple planning tools during initial project stages. The system relies on public data combined with generalizable language understanding to generate useful estimates without needing full specifications or historical team data. The system provides significant value to startup and freelance operations because these environments often do not have established formal processes or historical data.

The system generates both actionable subtasks and code through its unique functionality. Project teams can start their development work by using the output as a draft for their project. The system functions as both a project planning tool and a development assistance platform. The model demonstrates potential to become a fundamental tool for pre-project scoping, backlog grooming, onboarding and cost estimation which represent essential digital project management concerns.

## **Model Refinement and Future Enhancements**

Multiple model improvements emerged from the observed limitations of the system. The system requires improved scope control to prevent generating excessive unnecessary subtasks. This can be achieved through:

- Prompt tuning for MVP constraints. The MVP stands for Minimum Viable Product, Focus only on core functionality, avoid suggesting non critical subtasks, stay aligned with agile or lean development goals.
- Granularity control flags (e.g., toggle high level vs. detailed view).
- The use of domain specific LLMs provides consistent terminology and quality.
- A more structured output format (e.g., HTML with collapsible sections).

The performance of effort calibration would benefit from time tracked OSS issues instead of project metadata for better performance improvement. The system enables the estimation of effort for each subtask which leads to improved time forecasting at the sprint level for better planning accuracy. Finally, broader dataset enrichment (e.g., scraping JIRA boards or Trello cards from public agile projects) could provide a richer base for analogical reasoning.

## **Conclusions**

In conclusion, the XPERT model fulfills its objective as a hybrid, explainable, and semantically aware effort estimator for software projects. It offers clear advantages over legacy models in early stage planning, especially in agile and unstructured environments. Its integration of semantic matching, complexity analysis, and generative planning might position it as a novel contribution in the domain of AI assisted project management.

The system demonstrates practical capabilities for automated reasoning over software descriptions despite its limitations in various use cases. XPERT has the potential to become a core component of future project estimation platforms through additional tuning and validation which would enable it to bridge the idea to implementation gap at a cost effective and scalable level.

# Summary and Suggestions for Future Research

This research introduces an extensive approach to software effort estimation by developing and empirically evaluating XPERT model which is a hybrid AI powered pipeline that converts software task descriptions written in natural language into detailed development plans and also present time estimates. The research project and seminar included all stages of the research cycle starting from literature review, model conceptualization through to model implementation and testing as well as suggested critical evaluation of the model in real-world software scenarios.

The system incorporates multiple advanced functionalities which include semantic similarity retrieval via SBERT along with code complexity analysis, LLM based task decomposition and automated code generation to create structured person hour software effort estimates. The system underwent testing with various input data scenarios which showed its ability to produce full work plans and relevant code snippets together with effort estimates that use information from similar open-source repositories.

Digital project management may benefit significantly from this work through its direct application in early phase estimation and planning. Startups and freelance development teams along with rapid prototyping groups who lack historical data or mature estimation protocols can use XPERT to automatically evaluate their project scope and complexity. The model helps to clarify unclear project duration and description breakdown questions that are essential yet often resolved through unstructured or irregular methods. Through this research semantic AI tools achieve better understanding about their capability to connect theoretical project concepts with actual implementation schedules.

The project contributes to Software Effort Estimation (SEE) methodological innovation because it shows how current LLMs can be integrated with traditional models like PERT to generate understandable and flexible predictions. The research provides an operational framework for constructing hybrid estimation models which depend on neither code measurements nor historical records nor task repositories with labels.

The research contains specific constraints in its current state. The model has not received verification through real project timelines and developer logs in its current state. The code generated by the system functions as a reference but it remains non-production quality and does not integrate or test properly. The model's scope control system requires development because output results can become too extensive while failing to match minimal viable product targets. The OSS project retrieval method using SBERT may only show superficial matches because repository descriptions tend to be short or unclear.

For future research, several directions are proposed:

1. Per subtask effort estimation, calibrated using real-world issue tracking data (e.g., from GitHub, JIRA).
2. Interactive user feedback integration, enabling model retraining or refinement based on project team inputs.
3. Domain specific tuning of the LLM prompts for improved precision and reduced verbosity.
4. Application in enterprise project management platforms, such as integration with tools like Jira, Monday, or Azure DevOps.
5. Additional OSS repositories should be scanned and for speed of process all repositories should go through the complexity evaluator to get the effort estimation in advance. Of course that in this way, the parameters used to calculate the complexity to effort can't be modified, but if this is set in advance it will enable the XPERT solution to remain isolated from internet connection and therefore pass security regulation specifically in enterprise and governmental organizations.
6. Using company internal code repositories to evaluate and compare.
7. Implement additional "expert defined LLMs" to enrich the summary.pdf with additional important points.

These directions will help advance XPERT into practical, adaptive solutions that support human decision making in dynamic and complex software projects.

# Datasets references

## **GitHub Open-Source Repository Metadata (Custom Crawl)**

A curated dataset of approximately 12,000+ open-source software repositories collected from GitHub. The metadata included repository names, descriptions, README content, primary language, and contributor activity. This dataset was used to create a semantic index for retrieving similar projects based on task descriptions using SBERT embeddings.

See appendix B: collecting description and additional metadata from Github repositories

# References

1. Abbad-Andaloussi, A. (2023). On the relationship between source-code metrics and cognitive load: A systematic tertiary review. *Journal of Systems & Software*, 198, 111619.
2. Abrahao, S., De Marco, L., Ferrucci, F., Gomez, J., Gravino, C., & Sarro, F. (2018). Definition and evaluation of a COSMIC measurement procedure for sizing Web applications in a model-driven development environment. *Information and Software technology*, 104, 144-161.
3. Ahmed, M., Ibrahim, N. B., Nisar, W., Ahmed, A., Junaid, M., Soriano Flores, E., & Anand, D. (2024). A hybrid model for improving software cost estimation in global software development. *Computers, Materials & Continua*, 78(1), 1399-1422.
4. Alsheikh, N. M., & Munassar, N. M. (2023). Improving software effort estimation models using grey wolf optimization algorithm. *IEEE Access*, 11, 143549-143579.
5. Azzeh, M., Bou Nassif, A., & Attili, I. (2021). Predicting software effort from use case points: A systematic review. *Science of Computer Programming*, 204, 102596.
6. Azzeh, M., Bou Nassif, A., Elsheikh, Y., & Angelis, L. (2022). On the value of project productivity for early effort estimation. *Science of Computer Programming*, 219, 102819.
7. Bekamiri, H., Hain, D. S., & Jurowetzki, R. (2024). PatentSBERTa: A deep NLP based hybrid model for patent distance and classification using augmented SBERT. *Technological Forecasting and Social Change*, 206, 123536.
8. Butt, S. A., Khalid, A., Ercan, T., Ariza-Colpas, P. P., Melisa, A. C., Pineres-Espitia, G., ... & Ortega, R. M. (2022). A software-based cost estimation technique in scrum using a developer's expertise. *Advances in Engineering Software*, 171, 103159.
9. Butt, S. A., Ercan, T., Binsawad, M., Ariza-Colpas, P. P., Diaz-Martinez, J., Pineres-Espitia, G., ... & De-La-Hoz-Hernández, J. D. (2023). Prediction based cost estimation technique in agile development. *Advances in engineering software*, 175, 103329.
10. Campbell, G. A. (2018, May). Cognitive complexity: An overview and evaluation. In *Proceedings of the 2018 international conference on technical debt* (pp. 57-58).
11. Daniella, G., Wijaya, N., Putra, N. G. E., & Efata, R. (2023). Agile software development effort estimation based on product backlog items. *Procedia Computer Science*, 227, 186-193.
12. de Barros, G. R. C., & Martins, M. O. Comparative of source code generated by principal LLM generators for Python and Lua languages.

13. Ersoy, P., & Ersahin, M. (2024). Benchmarking LLaMA 3 70B for code generation: A comprehensive evaluation. *Orclever Proceedings of Research and Development*, 4(1), 52–58.
14. Favero, E. M. B., Casanova, D., & Pimentel, A. R. (2022). SE3M: A model for software effort estimation using pre-trained embedding models. *Information and Software Technology*, 147, 106886.
15. Feitelson, D. G. (2023). From code complexity metrics to program comprehension. *Communications of the ACM*, 66(5), 48–57.
16. Iqbal, M., Ijaz, M., Mazhar, T., Shahzad, T., Abbas, Q., Ghadi, Y., ... & Hamam, H. (2024). Exploring issues of story-based effort estimation in Agile Software Development (ASD). *Science of Computer Programming*, 236, 103114.
17. Jorgensen, M., & Halkjelsvik, T. (2020). Sequence effects in the estimation of software development effort. *Journal of Systems and Software*, 159, 110448.
18. Kapur, R., & Sodhi, B. (2022). OSS effort estimation using software features similarity and developer activity-based metrics. *ACM Transactions on Software Engineering and Methodology*, 31(2), 33.
19. Kaur, L., & Mishra, A. (2019). Cognitive complexity as a quantifier of version-to-version Java-based source code change. *Information and Software Technology*, 106, 31–48.
20. Lavazza, L., & Robiolo, G. (2010). The role of the measure of functional complexity in effort estimation. *PROMISE '10: Proceedings of the 6th International Conference on Predictive Models in Software Engineering*, 1–9.
21. Lavazza, L., Liu, G., & Meli, R. (2020). Using extremely simplified functional size measures for effort estimation: An empirical study. *ESEM '20: Proceedings of the ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 1–10.
22. Lavazza, L., Abualkishik, A. Z., Liu, G., & Morasca, S. (2023). An empirical evaluation of the “Cognitive Complexity” measure as a predictor of code understandability. *Journal of Systems and Software*, 197, 111561.
23. Ma, X., Wang, L., Yang, N., Wei, F., & Lin, J. (2024, July). Fine-tuning llama for multi-stage text retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 2421-2425).
24. McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, (4), 308-320.
25. Misic, V. B., & Tesic, D. N. (1998). Estimation of effort and complexity: An object-oriented case study. *The Journal of Systems and Software*, 41(2), 133–143.

26. Phan, A. V., Chau, P. N., Nguyen, M. L., & Bui, L. T. (2018). Automatically classifying source code using tree-based approaches. *Data & Knowledge Engineering*, 114, 12–25.
27. Phannachitta, P. (2020). On an optimal analogy-based software effort estimation. *Information and Software Technology*, 125, 106330.
28. Pospieszny, P., Czarnacka-Chrobot, B., & Kobylinski, A. (2018). An effective approach for software project effort and duration estimation with machine learning algorithms. *Journal of Systems and Software*, 137, 184–196.
29. Qi, F., Jing, X.-Y., Zhu, X., Xie, X., Xu, B., & Ying, S. (2017). Software effort estimation based on open source projects: Case study of GitHub. *Information and Software Technology*, 92, 145–157.
30. Rosa, W., & Jardine, S. (2025). A novel method to early agile effort estimation through functional initiatives. *Journal of Systems and Software*, 223, 112302.
31. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *arXiv preprint arXiv:1908.10084*.
32. Rijwani, P., & Jain, S. (2016). Enhanced software effort estimation using multi layered feed forward artificial neural network technique. *Procedia Computer Science*, 89, 307–312.
33. Rashid, C. H., Shafi, I., Khattak, B. H. A., Safran, M., Alfarhood, S., & Ashraf, I. (2025). ANN-based software cost estimation with input from COCOMO: CANN model. *Alexandria Engineering Journal*, 113, 681–694.
34. Singal, P., Kumari, A. C., & Sharma, P. (2020). Estimation of software development effort: A differential evolution approach. *Procedia Computer Science*, 167, 2643–2652.
35. Shin, J., Tang, C., Mohati, T., Nayebi, M., Wang, S., & Hemmati, H. (2025). Prompt engineering or fine-tuning: An empirical assessment of large language models for code generation. *arXiv preprint arXiv:2310.10508*.
36. Sharma, A., & Chaudhary, N. (2023). Prediction of Software Effort by Using Non-Linear Power Regression. *Procedia Computer Science*, 218, 1601–1611.
37. Sousa, A. O., Veloso, D. T., Gonçalves, H. M., Faria, J. P., Mendes-Moreira, J., Graça, R., ... & Henriques, P. C. (2023). Applying machine learning to estimate the effort and duration of individual tasks in software projects. *IEEE Access*, 11, 89933–89946.
38. Usman, M., Britto, R., Damm, L. O., & Borstler, J. (2018). Effort estimation in large-scale software development: An industrial case study. *Information and Software Technology*, 99, 21–40.
39. Wysocki, W., Miciuła, I., & Mastalerz, M. (2022). Classification of Task Types in Software Development Projects. *Electronics* 2022, 11, 3827.
40. Wysocki, W. (2023). Task planning model of software process. *Procedia Computer Science*, 225, 736–745.



41. Zare, F., Zare, H. K., & Fallahnezhad, M. S. (2016). Software effort estimation based on the optimal Bayesian belief network. *Applied Soft Computing*, 49, 968–980.

# Appendices

## Appendix A

### **The business approach: How similar tools are used in the industry these days**

Project management tools have achieved higher efficiency through artificial intelligence (AI) integration during the last few years. Atlassian Jira implements Atlassian Intelligence to deliver automated task management and predictive analytics and workload optimization features for team operations (Atlassian, 2024). The AI capabilities of ClickUp enable task decomposition and automated generation of actionable items from unstructured input and project discussion summarization which improves productivity and team coordination (ClickUp, 2024). The AI-first platform from Monday.com enables real-time risk detection and automatic resource distribution for tasks which optimizes project execution (Monday.com, 2024). Microsoft's Copilot uses large language models (LLMs) to create task plans and determine durations and distribute workloads dynamically within its project operations suite (Microsoft, 2024).

AI applications reach further than resource and task management because they enable process bottlenecks detection through process mining and simulation for workflow optimization. Celonis implements AI-based process mining to examine operational data which generates process models that detect operational inefficiencies in real time (Celonis, 2024). The ProM framework provides research grade process mining capabilities through its extensive range of plug-ins that enables thorough analysis of process execution logs (ProM, 2024). AnyLogic and Simio provide simulation tools which offer advanced modeling platforms that enable agent based and discrete event and system dynamics simulations for project workflow stress testing and outcome forecasting (AnyLogic, 2024; Simio, 2024). Resource Guru and Toggl Plan implement AI scheduling systems to track resource availability which enables organizations to optimize resource use without exceeding capacity limits (Resource Guru, 2024; Toggl Plan, 2024).

#### Web references - Current industry use of similar tools

- Atlassian. (2024). *Atlassian Intelligence for Jira*. <https://www.atlassian.com/platform/artificial-intelligence>
- ClickUp. (2024). *How to Use AI in Project Management*. <https://clickup.com/blog/how-to-use-ai-project-management>
- Monday.com. (2024). *AI-powered Work Management*. <https://monday.com/w/ai>
- Microsoft. (2024). *Copilot features in Project Operations*. <https://learn.microsoft.com/en-us/dynamics365/project-operations/project-management/copilot-features>
- Celonis. (2024). *What is Process Mining?*. <https://www.celonis.com/process-mining/what-is-process-mining>

- ProM Tools. (2024). *Process Mining Framework*. <https://promtools.org>
- AnyLogic. (2024). *Simulation Modeling Software*. <https://www.anylogic.com>
- Simio. (2024). *Simulation Software for Digital Twin and Optimization*. <https://www.simio.com>
- Resource Guru. (2024). *Resource Scheduling Software*. <https://resourceguruapp.com>
- Toggl Plan. (2024). *Resource Planning Tool*. <https://toggl.com/plan/use-cases/resource-planning>

## Appendix B

### Collecting description and additional metadata from Github repositories

#### Overview of `crawler-for-all-repositories.py`

[The following is an auto generated description of python code file]

This section describes the functionality of the `crawler-for-all-repositories.py` script, which fetches repository details from GitHub using both the REST and GraphQL APIs and stores the information in an SQLite database.

#### 1. Script Components

##### 1.1 Imports

The script imports several Python modules for handling API requests, database operations, authentication, and time-related functions:

- `sqlite3` for interacting with an SQLite database.
- `requests` for making HTTP requests to the GitHub API.
- `os` for handling file paths.
- `pandas` for working with data in tabular format.
- `getpass` for securely retrieving the GitHub token.
- `time` for handling API rate limits and time conversions.

##### 1.2 GitHub API Configuration

- The script defines the base URLs for the GitHub REST and GraphQL APIs.
- It prompts the user to enter a GitHub personal access token (`github_token`) for authentication.
- Headers for API requests include the authentication token and the appropriate `Accept` header for GitHub API versioning.

#### 2. Functions

##### 2.1 `check_rate_limit()`

- Checks the remaining API rate limits using GitHub's `/rate_limit` endpoint.
- Displays the number of remaining requests and the reset time.
- Returns the number of remaining requests.

## 2.2 fetch\_repos\_graphql(repos, batch\_size)

- Fetches repository details using the GitHub GraphQL API.
- Queries repository names, owners, descriptions, disk usage, and primary programming languages.
- Handles API rate limits by reducing batch size and retrying if needed.
- Returns a dictionary with repository data.

## 2.3 process\_repositories(cursor, repositories, limit=None)

- Processes repositories by retrieving their details using fetch\_repos\_graphql().
- Inserts fetched repository metadata (owner, repo name, description, size, and languages) into an SQLite database.
- Limits the number of processed repositories if specified.

## 2.4 display\_table\_samples(conn)

- Retrieves and prints the first 5 rows from each table in the SQLite database for validation.

## 3. Main Execution (main(mode))

### 3.1 Database Setup

- Defines paths for the cleaned SQL database and a new database with descriptions.
- Loads an SQLite database schema from an external SQL file (sdee\_lite\_cleaned.sql).
- Creates a new table repo\_additional\_info to store repository metadata.

### 3.2 Fetching and Storing Repository Data

- Queries repository names and owners from an existing table (repo\_info\_pv\_vec).
- Calls process\_repositories() to fetch repository details and store them in SQLite.
- Saves the updated database to sdee-with-description/sdee\_lite\_description.db.
- Exports the new dataset schema to a SQL script (sdee\_lite\_description.sql).

### 3.3 Displaying Data

- Calls display\_table\_samples() to show sample repository records from the SQLite database.

## 4. Script Execution

The script runs in either:

- Test mode (mode="test") to process only 20 repositories.
- Full mode (mode="all") to process all repositories.

The script execution starts with:

```
if __name__ == '__main__':  
    mode = "all"  
    main(mode)
```

## 5. Summary

1. Connects to an SQLite database and loads an existing schema.
2. Fetches repository metadata from GitHub via GraphQL.
3. Stores the fetched metadata in a new table.
4. Saves the updated database and exports an SQL script.
5. Displays sample data for validation.

## Appendix B'

### OSS Effort Estimation - description field Data Cleaning

This section describes all the data cleaning stages applied to the repo\_additional\_info dataset before estimating software development effort. The cleaning process ensures only meaningful, high-quality, English textual data is preserved.

#### Overview

- Input Source: sdee\_lite\_description.sql
- Target Table: repo\_additional\_info

#### Output Files:

- Cleaned SQL Dump: sdee\_lite\_description\_cleaned.sql
- Cleaning Log: removed-records.log

#### Cleaning Stages

1. Load & Normalize Input
  - Load the SQLite dump into memory.
  - Ensure description is treated as a string.
  - Strip whitespace.
2. Remove Records with description = 'None'
  - Case-insensitive match for: "None", Python None
3. Remove description that is "null" or variations
  - Matches: "NULL", "null", [null], [NULL], etc.
  - Converted to NaN and dropped later as null.
  - Missing (NaN) or Empty string after stripping or Whitespace only
4. Remove Descriptions Exceeding 3500 Characters
  - Such descriptions are unusually long and likely contain non-relevant or auto-generated content, or in some cases hold wrong information belonging to other Github fields, which we presume had some kind of bug.
5. Remove Records with Forbidden Terms
  - Remove Records with Specific "Unmaintained" Text - Specifically: "This project is not maintained anymore" (case-insensitive)
  - If description contains:  
[DEPRECATED], READ-ONLY, THIS REPO IS UNMAINTAINED, UNMAINTAINED, ABANDONED, etc.  
Regex is case-insensitive and removes surrounding brackets.  
Statistics are tracked before and after cleaning and saved to the cleaning run log and to the removed records log.
6. Remove Emojis and Special Symbols
  - All Unicode emoji blocks are removed using a regex.  
Covers: Flags, Symbols, Emoticons (e.g., , , , , etc.)
7. Remove Non-English Words

- Keeps only words matching [a-zA-Z]{2,} regex
  - Removes any bracketed content (e.g., Chinese/Japanese inside brackets).  
Example cleaned: "中文自然语言处理 Toolkit for NLP" -> "Toolkit for NLP"
8. Normalize Unicode
    - Converts all descriptions to a canonical Unicode format (NFKC).
    - Prevents issues caused by invisible, composite, or unusual characters.
  9. Synchronize Removals Across Tables
    - When a record is removed from repo\_additional\_info, it is also deleted from all other tables.
  10. Update Cleaned Descriptions into SQLite
    - All remaining (non-removed) cleaned descriptions are written back to the repo\_additional\_info table in the SQLite database.

## Appendix C

### Utilizing SBERT for Semantic Analysis of Software Descriptions

#### SBERT: Converting Text to Vectors

[The following is an auto generated description out of python code file]

SBERT processes software descriptions and generates dense vector embeddings, placing semantically similar descriptions closer in the vector space. This facilitates improved software similarity detection.

*Example Python code:*

```
from sentence_transformers import SentenceTransformer

# Load SBERT model
sbert_model = SentenceTransformer("all-MiniLM-L6-v2")

# Example software description
description = "A lightweight AI-based image compression tool"

# Convert to SBERT vector
vector = sbert_model.encode(description)

print(vector.shape) # Output: (384,)
print(vector[:5])  # Output: First 5 values of the vector
```

In this example, the “**all-MiniLM-L6-v2**” model produces a 384-dimensional vector representing the semantic content of the description.

### Advantages Over Traditional Methods

- **Semantic Understanding:** Unlike TF-IDF or Word2Vec, SBERT captures the context and meaning of entire sentences, leading to more accurate similarity assessments.
- **Efficiency:** SBERT provides quick computation of embeddings, making it suitable for large-scale software repositories.
- **Improved Similarity Detection:** By understanding the semantics, SBERT enhances the detection of similar software projects, even when descriptions use different terminologies.

## Appendix D

### Using PERT and the Beta Distribution for Software Effort Estimation

[The following is an auto generated description]

To handle the inherent uncertainty in software effort estimation, we adopt the Program Evaluation and Review Technique (PERT) in combination with the Beta distribution. This hybrid approach provides a probabilistic boundary around effort predictions, rather than relying on a single deterministic value.

What is PERT?

PERT estimates expected effort using three expert-provided values:

- **Optimistic (O):** The best-case scenario (minimal effort).
- **Most Likely (M):** The most probable estimate.
- **Pessimistic (P):** The worst-case scenario (maximum effort).

PERT Formula:  $E = (O + 4M + P) / 6$

Standard Deviation (SD) and Variance:  $SD = (P - O) / 6$ ,  $Variance = ((P - O) / 6)^2$

Effort Range (95% Confidence Interval):  $[E - 2SD, E + 2SD]$

Why Use This Method?

- Quantifies uncertainty using a structured formula.
- Easy to incorporate expert knowledge via O, M, P values.
- Compatible with machine learning and static analysis features.
- Supports transparent stakeholder communication via effort ranges.

## Appendix E

### Running the solution: snapshots and example run log

When triggering the `./run.sh` script to run all 8 servers/services, from which “code-generator” has 8 workers, hence it runs additional 8 server processes.

The script first checks for “left-overs” running processes already using the required ports, and if used, it terminates each with Linux kill command.

```
(.venv) yoav-levinger:~/projects/source/2nd/see/llm-pipeline$ ./run-all.sh
🔪 Killing process on port 8002 (PID: 237473)
297264
297265
297266
297267
297268
297269
297270
297271)
Starting task-splitter on port 8001...
Starting code-generator on port 8002...
Starting tool-x-connector on port 8003...
Starting document-creator on port 8004...
Starting expert-advisor on port 8005...
Starting sbert-complexity-estimator on port 8006...
Starting main-controller on port 8080...
Starting frontend on port 8090...
✓ All services are starting. Press Ctrl+C to stop them.
INFO:   Uvicorn running on http://127.0.0.1:8002 (Press CTRL+C to quit)
INFO:   Started parent process [387595]
INFO:   Started server process [387596]
INFO:   Waiting for application startup.
INFO:   Application startup complete.
INFO:   Uvicorn running on http://127.0.0.1:8003 (Press CTRL+C to quit)
INFO:   Started server process [387600]
INFO:   Waiting for application startup.
INFO:   Application startup complete.
INFO:   Uvicorn running on http://127.0.0.1:8004 (Press CTRL+C to quit)
INFO:   Started server process [387601]
INFO:   Waiting for application startup.
INFO:   Application startup complete.
INFO:   Uvicorn running on http://127.0.0.1:8005 (Press CTRL+C to quit)
INFO:   Started server process [387594]
INFO:   Waiting for application startup.
INFO:   Application startup complete.
INFO:   Uvicorn running on http://127.0.0.1:8006 (Press CTRL+C to quit)
INFO:   Started server process [387609]
INFO:   Waiting for application startup.
```





## eXpert Planning and Estimation using Retrieval and Transformers

### Submit Your Software Task

Software Description:

Select Expert Advisors:

- ☒ Security
- ☐ Accessibility
- ☐ Performance

Upload Custom Policy Files (optional):

Choose Files

 No file chosen
 

 Run Pipeline

Basic user interface that enables: Typing in the description of the software, Select “Expert Advisors”, uploading document guidelines such as security rules.

Additional option is to use API call to trigger the main controller directly, and as we have tested the solution, we used curl from the command line:

```
yoav-levinger:~$ curl -X POST http://localhost:8080/process \
-H "Content-Type: application/json" \
-d '{"description": "Build a web dashboard for monitoring system metrics and user login activity"}'
```

And also run multiple tests, using scripts such as this, with total run duration of about 9.5 hours when defined with 16 workers (indeed a long-run without memory leaks):

```
#!/bin/bash
# Endpoint
URL="http://localhost:8080/process"

# List of 32 software task descriptions
DESCRIPTIONS=(
  "Build a REST API for a to-do list application"
  "Create a real-time chat application using websockets"
```

```

"Develop a single-page app for online shopping"
"Implement a file upload and processing backend"
"Build a dashboard for IoT sensor data monitoring"
"Create an OAuth2 login system with Google and GitHub"
"Develop a CI/CD pipeline using GitHub Actions"
"Create an admin panel for a blogging platform"
"Implement a PDF report generator for time logs"
"Build a static site generator in Python"
"Develop an e-commerce shopping cart with checkout"
"Create a scheduler for employee shifts"
"Build a Slack bot for task reminders"
"Develop a system for image classification using CNN"
"Create an API gateway with rate limiting"
"Implement a job queue and worker system"
"Build a microservice for currency conversion"
"Create a service that scrapes news headlines daily"
"Develop a webhook listener for payment notifications"
"Create a CLI tool for GitHub issue tracking"
"Build a plugin-based architecture for data exporters"
"Develop a testing framework for backend APIs"
"Create a dynamic form generator from JSON schema"
"Build a serverless function to resize images"
"Create a user permission and roles service"
"Develop a crypto price tracker and alert system"
"Implement a notification system with fallback SMS"
"Build a marketplace for freelance developers"
"Create a RESTful API for managing Kubernetes clusters"
"Develop a browser extension for blocking ads"
"Build a chatbot that summarizes articles"
"Create an analytics service for clickstream data"
)

# Loop through and send POST request for each description
for i in "${!DESCRIPTIONS[@]}; do
  echo "🔄 Request $((i+1)):"
  curl -s -X POST "$URL" \
    -H "Content-Type: application/json" \
    -d "{\"description\": \"${DESCRIPTIONS[$i]}\"}" \
    | jq '.'
  echo "-----"
done

```

For the scope of this project, all services run locally and use CPU. Even the LLM used are configured to use CPU, so when activity starts, we can see all the CPU reaching high usage.



## Appendix F

### The Interquartile Range (IQR) method

As used in the “effort\_estimator\_combined.py” file in “remove\_outliers” function:

```
def remove_outliers(results: List[Dict], key: str = "hours") -> List[Dict]:
    values = [r[key] for r in results if key in r]
    if not values:
        return results
    q1, q3 = np.percentile(values, [25, 75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    filtered = [r for r in results if lower <= r[key] <= upper]
    return filtered if filtered else results # fallback if all removed
```

#### What it does:

- It takes a list of effort results (each with a "hours" field).
- Computes the **25th (Q1)** and **75th (Q3)** percentiles.
- Calculates the **interquartile range**:  $IQR = Q3 - Q1$ .
- Defines bounds:
  - Lower bound =  $Q1 - 1.5 \times IQR$
  - Upper bound =  $Q3 + 1.5 \times IQR$
- Filters out any repositories with effort estimates outside this range.

This ensures that extreme values (too low or too high) don't skew the average effort computed for semantically similar repositories.

## Appendix G

### Nonlinear LOC-Based Modifier

Inside `EffortEstimator.calculate_effort()` in `code_effort_estimator.py`:

```
# LOC adjustment - adds more time if the repo is large
loc = self.metrics.loc
loc_adjustment = (loc ** 0.5) * 0.03 # nonlinear, increases more naturally
adjusted_effort_days = effort_days + loc_adjustment
```

What it does:

This modifier **adds additional effort** (in days) based on the **size of the codebase** (LOC), using a **square root transformation**:

$$Effort_{adj} = \sqrt{LOC} \times 0.03$$

This formula is:

- **Nonlinear**: grows slowly with small LOC, but more steeply as size increases
- **Smooth**: avoids harsh jumps or step functions
- **Empirically designed**: based on observed effort estimates across many OSS repos

Why it's needed:

The main complexity model already includes LOC as a factor, but this **modifier adds a second-level correction**, ensuring that:

- Very large codebases still get a proportionate bump
- Small codebases don't get overestimated by metrics like cyclomatic complexity alone

## Appendix H

### Explanation of Code-Based Effort Estimation Formula and Implementation in XPERT while presenting the actual code implementation sections from our solution

Implementation details and theoretical foundation of the code-based software effort estimation model used in the XPERT system. The model calculates effort estimates using a composite complexity score derived from static code analysis, which is then wrapped in a PERT estimation formula. It includes specific metrics, weights, and normalization steps and is implemented primarily in `code_effort_estimator.py` and orchestrated via the `effort_estimator_combined.py` API.

#### 1. Composite Complexity Score (C\_comp)

The composite score is calculated using six base metrics extracted from the codebase:

| Metric                | Symbol | Description                               |
|-----------------------|--------|-------------------------------------------|
| Lines of Code         | LOC    | Logical lines of code                     |
| Cyclomatic Complexity | CC     | Average control flow complexity           |
| Halstead Volume       | HV     | Measures size based on operators/operands |
| Cognitive Complexity  | CoCo   | Human readability and mental effort       |
| Number of Functions   | NF     | Total number of functions/methods         |
| AST Depth             | ASTd   | Average depth of Abstract Syntax Tree     |

These are aggregated using the following formula:

$$C\_comp = w1 * LOC\_norm + w2 * CC\_norm + w3 * HV\_norm + w4 * CoCo\_norm + w5 * \log(NF + 1) + w6 * ASTd\_norm$$

Default weights as used in the code - after empirical evaluations  
(EffortEstimator.calculate\_composite\_complexity):

| Weight | Metric | Value |
|--------|--------|-------|
| w1     | LOC    | 6.5   |
| w2     | CC     | 1.2   |
| w3     | HV     | 1.2   |
| w4     | CoCo   | 1.0   |
| w5     | NF     | 2.0   |
| w6     | ASTd   | 1.0   |

### Implementation:

- Metric extraction via CodeMetrics.analyze()
- Composite score via EffortEstimator.calculate\_composite\_complexity()

Metric Evaluation Details (with code quotes)

### Lines of Code (LOC):

```
self.loc += sum(1 for line in source.splitlines() if line.strip())
```

### Cyclomatic Complexity (CC):

```
def _calculate_cyclomatic_complexity(self, tree: ast.AST) -> int:
    return 1 + sum(1 for node in ast.walk(tree)
                    if isinstance(node, (ast.If, ast.For, ast.While, ast.Try, ast.With, ast.And, ast.Or)))
```

### Halstead Volume (HV):

```
def _approximate_halstead_volume(self, source: str) -> float:
    tokens = source.split()
    ops = [tok for tok in tokens if tok in ['+', '-', '*', '/', '%', '=', '==', '!=', '<', '>', '>=', '<=']]
    opr = [tok for tok in tokens if tok not in ops]
    n1, n2 = len(set(ops)), len(set(opr))
    N1, N2 = len(ops), len(opr)
    return (N1 + N2) * math.log2(n1 + n2) if (n1 + n2) > 0 else 0
```

### Cognitive Complexity (CoCo):

```
def _approximate_cognitive_complexity(self, tree: ast.AST) -> int:
    def _score(node, depth=1):
        if isinstance(node, (ast.If, ast.While, ast.For, ast.With, ast.FunctionDef)):
            return depth + sum(_score(child, depth + 1) for child in ast.iter_child_nodes(node))
```

```

    return sum(_score(child, depth) for child in ast.iter_child_nodes(node))
return _score(tree)

```

### Number of Functions (NF):

```

self.function_count += len([n for n in ast.walk(tree) if
isinstance(n, ast.FunctionDef)])

```

and counted in Lizard analysis with:

```

self.function_count += 1

```

### AST Depth (ASTd):

```

def _calculate_ast_depth(self, node: ast.AST, level: int = 0) ->
int:
    if not list(ast.iter_child_nodes(node)):
        return level
    return max(self._calculate_ast_depth(child, level + 1) for child
in ast.iter_child_nodes(node))

```

## 2. PERT-Based Effort Estimation

Once C\_comp is calculated, it is passed into a PERT function:

```

Optimistic = 0.85 * C_comp
Most_Likely = C_comp
Pessimistic = 2.0 * C_comp
Effort = (O + 4M + P) / 6

```

### Most\_Likely = C\_comp

This is directly from the calculated complexity score.

### Optimistic = 0.85 \* C\_comp

This reflects an optimistic outcome (15% quicker) under ideal conditions: good documentation, low coupling, experienced developers.

### Pessimistic = 2.0 \* C\_comp

Doubles the complexity to account for risks like refactoring, misunderstood requirements, or integration problems.

This yields an effort value in **days**, which is then multiplied by **8** to get **hours**.



### 3. Local vs Analogical Estimation

`effort_estimator_combined.py` implements a dual estimation pipeline:

#### 1. Analogical Estimation:

- Retrieves top-k similar repositories using SBERT via `get_top_k_similar_repos()` from `get_similarity_repositories_sbert.py`
- Each is cloned and evaluated via `evaluate_multiple_repos()` from `github_repo_complexity_evaluator_multiple_to_see.py`
- Outliers are removed via IQR filtering (`remove_outliers()`)

#### 2. Local Estimation:

- Local generated code folder is evaluated via `evaluate_codebase()`

#### Result:

- A **CombinedEstimationResponse** containing both GitHub baseline and local effort in hours.

### 4. Language Normalization (Heuristics)

To account for verbosity differences between languages, a **LANGUAGE\_WEIGHTS** dictionary adjusts the final effort (e.g., Java = 0.9, C++ = 1.1).

# Appendix I

## Sample backend run log for effort estimating Github repository

### Stages

- Cloning a Github repository
- Run the code complexity analyzers
- Present the summary metrics

Some of these results will eventually appear in the Summary.pdf file

```
📦 Cloning https://github.com/flask-restful/flask-restful.git to
/tmp/tmpkso2q479/flask-restful
Cloning into '/tmp/tmpkso2q479/flask-restful'...
remote: Enumerating objects: 3622, done.
remote: Counting objects: 100% (30/30), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 3622 (delta 20), reused 6 (delta 6), pack-reused 3592 (from 2)
Receiving objects: 100% (3622/3622), 1.10 MiB | 4.17 MiB/s, done.
Resolving deltas: 100% (2432/2432), done.
Analyzing codebase at: /tmp/tmpkso2q479/flask-restful
🔍 [unknown] Analyzing: /tmp/tmpkso2q479/flask-restful/.coveragerc
🔍 [unknown] Analyzing: /tmp/tmpkso2q479/flask-restful/MANIFEST.in
🔍 [python] Analyzing: /tmp/tmpkso2q479/flask-restful/setup.py
🔍 [unknown] Analyzing: /tmp/tmpkso2q479/flask-restful/Makefile
🔍 [python] Analyzing: /tmp/tmpkso2q479/flask-restful/scripts/release.py
🔍 [python] Analyzing: /tmp/tmpkso2q479/flask-restful/flask_restful/inputs.py
🔍 [python] Analyzing: /tmp/tmpkso2q479/flask-restful/flask_restful/__init__.py
🔍 [python] Analyzing: /tmp/tmpkso2q479/flask-restful/flask_restful/reqparse.py
🔍 [python] Analyzing:
/tmp/tmpkso2q479/flask-restful/flask_restful/__version__.py
🔍 [python] Analyzing: /tmp/tmpkso2q479/flask-restful/flask_restful/fields.py
🔍 [python] Analyzing:
/tmp/tmpkso2q479/flask-restful/flask_restful/utils/__init__.py
🔍 [python] Analyzing:
/tmp/tmpkso2q479/flask-restful/flask_restful/utils/crypto.py
🔍 [python] Analyzing:
/tmp/tmpkso2q479/flask-restful/flask_restful/utils/cors.py
🔍 [python] Analyzing:
/tmp/tmpkso2q479/flask-restful/flask_restful/representations/__init__.py
🔍 [python] Analyzing:
/tmp/tmpkso2q479/flask-restful/flask_restful/representations/json.py
🔍 [python] Analyzing:
/tmp/tmpkso2q479/flask-restful/examples/xml_representation.py
🔍 [python] Analyzing: /tmp/tmpkso2q479/flask-restful/examples/todo_simple.py
```

```

[python] Analyzing: /tmp/tmpkso2q479/flask-restful/examples/todo.py
[python] Analyzing: /tmp/tmpkso2q479/flask-restful/tests/test_accept.py
[python] Analyzing: /tmp/tmpkso2q479/flask-restful/tests/test_cors.py
[python] Analyzing: /tmp/tmpkso2q479/flask-restful/tests/__init__.py
[python] Analyzing: /tmp/tmpkso2q479/flask-restful/tests/test_inputs.py
[python] Analyzing: /tmp/tmpkso2q479/flask-restful/tests/test_reqparse.py
[python] Analyzing: /tmp/tmpkso2q479/flask-restful/tests/test_api.py
[python] Analyzing:
/tmp/tmpkso2q479/flask-restful/tests/test_api_with_blueprint.py
[python] Analyzing: /tmp/tmpkso2q479/flask-restful/tests/test_fields.py
Summary Metrics:
- LOC: 4696
- Avg CC: 15.423076923076923
- Avg Halstead: 5422.357504236624
- Avg Cognitive: 49.42307692307692
- Avg AST Depth: 9.043478260869565
- Functions: 538
PERT-Based Effort Estimation:
  Optimistic: 15.51 | Most Likely: 21.25 | Pessimistic: 57.71 | Effort (days):
28.43
C_comp = 21.254
{'C_comp': 21.254, 'Optimistic': 15.51, 'Most Likely': 21.25, 'Pessimistic':
57.71, 'Effort (days)': 28.43, 'hours': 250.16}

Academic PERT-Based Estimation
Formula: Effort = PERT(0.85 * C_comp, C_comp, 2.0 * C_comp)
| Metric          | Value          |
|-----|-----|
| owner           | flask-restful |
| repo            | flask-restful |
| C_comp          | 21.254        |
| complexity_mode | academic      |
| Optimistic      | 15.51         |
| Most_Likely     | 21.25         |
| Pessimistic     | 57.71         |
| Effort (days)  | 28.43         |
| hours           | 227.44        |
Evaluating repository: votem/semaphore-service

```

## Appendix J

## Logs for running the code complexity estimator on our solution folder

```
./venv/bin/python code_effort_estimator.py
```

```
Running estimation on: /llm-pipeline
```

```
Analyzing codebase at: /llm-pipeline
```

```
[unknown] Analyzing: /llm-pipeline/run-all.sh
```

```
[python] Analyzing: /llm-pipeline/backend/document-creator/document_creator.py
```

```
[python] Analyzing: /llm-pipeline/backend/document-creator/ref-old/document_creator.py
```

```
[python] Analyzing: /llm-pipeline/backend/document-creator/ref-old/document_creator_old.py
```

```
[python] Analyzing: /llm-pipeline/backend/tool-x-connector/tool_x_connector.py
```

```
[python] Analyzing: /llm-pipeline/backend/main-controller/main_controller.py
```

```
[python] Analyzing: /llm-pipeline/backend/code-generator/code_generator.py
```

```
[python] Analyzing:
```

```
/llm-pipeline/backend/sbert_complexity_estimator/get_similarity_repositories_sbert.py
```

```
[python] Analyzing:
```

```
/llm-pipeline/backend/sbert_complexity_estimator/sbert_complexity_estimator.py
```

```
[python] Analyzing: /llm-pipeline/backend/sbert_complexity_estimator/code_effort_estimator.py
```

```
[python] Analyzing: /llm-pipeline/backend/sbert_complexity_estimator/test_repo_effort.py
```

```
[python] Analyzing:
```

```
/llm-pipeline/backend/sbert_complexity_estimator/github_repo_complexity_evaluator_multiple_t  
o_see.py
```

```
[python] Analyzing:
```

```
/llm-pipeline/backend/sbert_complexity_estimator/effort_estimator_combined.py
```

```
[python] Analyzing: /llm-pipeline/backend/sbert_complexity_estimator/test_repo_effort_01.py
```

```
[python] Analyzing: /llm-pipeline/backend/task-splitter/task_splitter.py
```

```
[python] Analyzing: /llm-pipeline/backend/shared/models.py
```

```
[python] Analyzing: /llm-pipeline/backend/expert-advisor/expert_advisor.py
```

```
[python] Analyzing: /llm-pipeline/frontend/frontend_app.py
```

```
[unknown] Analyzing: /llm-pipeline/tests/run_tests1.sh
```

```
[unknown] Analyzing: /llm-pipeline/tests/run_tests.sh
```

```
Summary Metrics:
```

```
- LOC: 1844
```

```
- Avg CC: 9.5
```

```
- Avg Halstead: 2697.194559233109
```

```
- Avg Cognitive: 18.45
```

```
- Avg AST Depth: 10.529411764705882
```

```
- Functions: 62
```

```
PERT-Based Effort Estimation:
```

```
Optimistic: 16.92 | Most Likely: 23.30 | Pessimistic: 63.86 | Effort (days): 30.29
```

```
===== Final Result Summary =====
```

```
C_comp: 23.305
```

Optimistic: 16.92  
Most Likely: 23.3  
Pessimistic: 63.86  
Effort (days): 30.29  
**hours: 266.54**

# Appendix k

## Independent Work Declaration

6

תקנון עבודת גמר תואר שני במערכות מידע

### נספח א: טופס הצהרה על מקוריות עבודת הגמר (למסירה עם הגשת העבודה הסופית)

עבודת הגמר נועדה לאפשר לך ליישם לקראת סיום התואר את הידע והמיומנות שרכשת במהלך הלימודים. בעבודה זו צריכים לבוא לכדי ביטוי כישורים אקדמיים כגון יכולת קריאה, הבנה וניתוח של טקסטים מדעיים, אינטגרציה של סוגי ידע שונים, חשיבה ביקורתית, כושר תכנון מחקר וביצועו וכתובה מדעית רהוטה.

עבודת הגמר חייבת להיות **יצירה עצמית שלך**. כדי להבחין בין תרומתך לבין תרומות של אחרים, וכדי למנוע פגיעה בקניין הרוחני של כותבים שעליהם נסמכת העבודה, מחובתך להקפיד על ציון המקורות שעליהם הסתמכת. בכלל זה ציטוטים ישירים של טיעונים או ממצאים, רעיונות, דעות ופרשנויות של אנשים אחרים. סטייה מכללי הציטוט, לא כל שכן נטילה ללא ציון ראוי של חלקי עבודה או עבודה שלמה של כותבים אחרים, מתפרשת לחומרה כניסיון להציג דברי אחרים כדברים שלך, והדבר מהווה עברה חמורה על כללי האתיקה המדעית.

יש לציין במפורש האם בוצעה שילוב של התוכן שנוצר באמצעות בינה מלאכותית יוצרת (מערכות כגון ChatGPT) וסימון התוכן בתוך הטקסט.

כדי למנוע אי הבנה בנדון אנו מבקשים ממך לחתום על ההצהרה הבאה:

אני, **יואב לוינגר**, ת.ז. **033829557** מצהיר/ה בזאת כי עבודת התזה/הגמר המצורפת בזאת היא **פרי יצירתי העצמית** ונכתבה על-פי כללי הציטוט וההפניה המקובלים באקדמיה. כמו כן, אני מצהיר/ה כי ידוע לי שהגשת עבודה אשר חלקים רבים ו/או משמעותיים ו/או מהותיים בה הועתקו מעבודה אחרת היא עברה וכי אם יתגלה כי עברתי עברה זו, תוגש נגדי תלונה על כך לוועדת המשמעת של המכללה האקדמית של תל אביב יפו.



חתימה

21/04/2025

תאריך

## נספח א: טופס הצהרה על מקוריות עבודת הגמר (למסירה עם הגשת העבודה הסופית)

עבודת הגמר נועדה לאפשר לך ליישם לקראת סיום התואר את הידע והמיומנות שרכשת במהלך הלימודים. בעבודה זו צריכים לבוא לכדי ביטוי כישורים אקדמיים כגון יכולת קריאה, הבנה וניתוח של טקסטים מדעיים, אינטגרציה של סוגי ידע שונים, חשיבה ביקורתית, כושר תכנון מחקר וביצועו וכתובה מדעית רהוטה.

עבודת הגמר חייבת להיות יצירה עצמית שלך. כדי להבחין בין תרומתך לבין תרומות של אחרים, וכדי למנוע פגיעה בקניין הרוחני של כותבים שעליהם נסמכת העבודה, מחובתך להקפיד על ציון המקורות שעליהם הסתמכת. בכלל זה ציטוטים ישירים של טיעונים או ממצאים, רעיונות, דעות ופרשנויות של אנשים אחרים. סטייה מכללי הציטוט, לא כל שכן נטילה ללא ציון ראוי של חלקי עבודה או עבודה שלמה של כותבים אחרים, מתפרשת לחומרה כניסיון להציג דברי אחרים כדברים שלך, והדבר מהווה עברה חמורה על כללי האתיקה המדעית.

יש לציין במפורש האם בוצעה שילוב של התוכן שנוצר באמצעות בינה מלאכותית יוצרת (מערכות כגון ChatGPT) וסימון התוכן בתוך הטקסט.

**כדי למנוע אי הבנה בנדון אנו מבקשים ממך לחתום על ההצהרה הבאה:**

אני טלי איציקוביץ' עברון, ת.ז. 200026243 מצהיר/ה בזאת כי עבודת התזה/הגמר המצורפת בזאת היא פרי יצירתי העצמית ונכתבה על-פי כללי הציטוט וההפניה המקובלים באקדמיה. כמו כן, אני מצהיר/ה כי ידוע לי שהגשת עבודה אשר חלקים רבים ו/או משמעותיים ו/או מהותיים בה הועתקו מעבודה אחרת היא עברה וכי אם יתגלה כי עברתי עברה זו, תוגש נגדי תלונה על כך לוועדת המשמעת של המכללה האקדמית של תל אביב יפו.



תאריך 21.4.25 חתימה